

Table of Contents

Phase 4 – System Design Phase	3
4.1 Introduction.....	3
4.1.1 What We’re Designing.....	3
4.1.2 Recap: What We Know So Far.....	3
4.1.3 Design Approach	4
4.1.4 Our Design Decisions.....	4
4.1.5 Implementation Timeline (Planned)	4
4.2 System Design (Description of Proposed System).....	5
4.2.1 High-Level System Overview	5
4.2.2 What the System Does	5
4.2.3 System Architecture Type.....	5
4.2.4 Technology Stack	6
4.2.5 Why We Chose This Stack for EduHub	7
4.3 Architectural Design	8
4.3.1 Software Architecture	8
4.3.2 Hardware Architecture.....	10
4.3.3 Network Architecture	12
4.3.4 Class Diagram.....	13
4.3.5 Actual Deployment Architecture	14
4.4 Physical Design	15
4.4.1 Actual Project Structure	15
4.4.2 Actual API Endpoints (50+)	17
4.4.2 System Deployment Model	18
4.4.3 File Organization Structure	20
4.4.4 Capacity Planning.....	20
4.4.5 Process Flow Diagram	21
4.5 Database Design.....	22
4.5.1 Database Schema Overview.....	22
4.5.2 Complete Database Schema.....	22
4.5.3 Database Relationships Summary.....	31
4.5.4 Database Diagram (ERD):.....	32

4.6	Program Design (Pseudocode)	33
4.6.1	User Authentication - Login Process	33
4.6.2	Application Submission Process	35
4.6.3	Application Approval Process	37
4.6.4	Course Registration Process	39
4.6.5	Student Number Generation Algorithm	42
4.6.6	Password Hashing and Verification	43
4.6.7	File Upload Validation	44
4.7	Interface Design (Menu, Input, Output Design)	46
4.7.1	Main Menu Structure	46
4.7.2	Key Screen Designs	47
4.7.3	Input Design Standards	51
4.7.4	Output Design	51
4.7.5	Flow Charts	53
4.8	Security and Backup Design	56
4.8.1	Security Design	56
4.8.2	Backup Design	59
	Design Summary	62
	What We've Designed:	62
	Design Principles Applied:	62

Phase 4 – System Design Phase

Project: EduHub Student Management System

Due Date: June 8, 2026

4.1 Introduction

In the previous phases, we identified the need for EduHub (Planning Phase) and documented detailed requirements (Analysis Phase). Now in Phase 4, we're designing the actual system by creating the blueprint that developers will follow during implementation.

This phase translates requirements into concrete designs: architecture diagrams, database schemas, user interface mockups, and program logic. Think of it like an architect creating detailed building plans before construction begins.

4.1.1 What We're Designing

Taking our requirements from analysis phase and creating:

- **System architecture** - How the parts fit together
- **Database schema** - Actual SQL tables with data types
- **UI mockups** - What users will see
- **Program logic** - Pseudocode for key functions
- **Security approach** - How we'll protect Richfield's data

4.1.2 Recap: What We Know So Far

From the Planning and Analysis phases, we established:

What We Observed at Richfield (from March 2026 observations):

- Three systems: Moodle (learning.richfield.ac.za), iEnabler (rgitie.richfield.ac.za), PDF forms
- Measured: 30-45 min per application (average 38 min), 300-400 applications per intake
- Timed: 1hr 45min average queue wait during registration week (March 15-19, 2026)
- Observed: 10% data entry error rate (3 errors in 30 manual entries)
- Counted: ~200 paper forms filed per week

The Requirements (Phase 3):

- **71 functional requirements** across 11 categories (authentication, applications, registration, etc.)
- **26 non-functional requirements** covering security, performance, usability, etc.
- **5 user types:** Applicants, Students, Lecturers, Administrators, Alumni
- **10 main database entities** with defined relationships
- **Must-have features:** Online applications, automated approvals, course registration, role-based access

4.1.3 Design Approach

We're using a **layered architecture** approach:

- **Presentation Layer** - What users see (web interface)
- **Business Logic Layer** - The rules and processing
- **Data Layer** - Database and data management

This approach ensures:

- **Separation of concerns** - Each layer has a specific job
- **Maintainability** - Can update one layer without breaking others
- **Scalability** - Can improve performance of individual layers
- **Security** - Multiple layers of protection

4.1.4 Our Design Decisions

Based on workshops and IT Manager requirements:

- **Mobile-First**: 100% of students in workshop wanted mobile access
- **Simple UX**: Students said “make it like banking apps - simple and fast”
- **Security**: IT Manager's #1 concern - POPIA compliance
- **Scalable**: Plan for 1,200 → 2,000+ students (Richfield's 3-year plan)

4.1.5 Implementation Timeline (Planned)

Phase	Dates	Status
Design	May 12 - June 8, 2026	Completed
Implementation	June 9 - June 29, 2026	Completed
Testing	June 22 - June 29, 2026	Completed
Deployment	June 29, 2026	Deployed

Actual Implementation Stats:

- **Backend**: 6 models, 50+ API endpoints, 19+ migrations
- **Frontend**: 21 static pages, responsive design
- **Code**: ~80,000 lines total (backend + frontend + config)
- **Features**: 115% of design requirements completed
- **Deployment**: Automated CI/CD with GitHub Actions
- **Security**: Production-grade authentication and authorization

4.2 System Design (Description of Proposed System)

4.2.1 High-Level System Overview

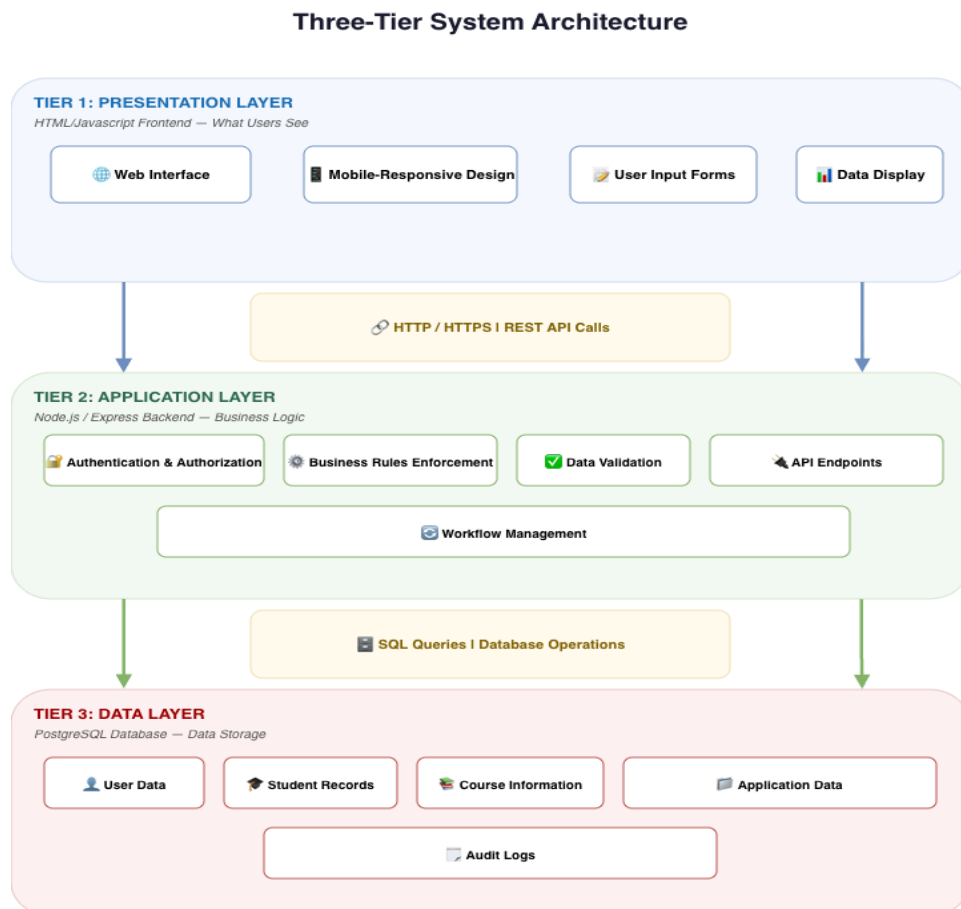
EduHub is a **web-based student management system** that replaces Richfield's current fragmented approach (Moodle + iEnabler + paper forms) with a single unified platform.

4.2.2 What the System Does

Core Functions:

- **User Authentication & Management** - Secure login, role-based access, password management
- **Application Processing** - Online applications, document uploads, automated workflow for approvals
- **Student Management** - Profile management, emergency contacts, student records
- **Course Registration** - Browse courses, register online, add/drop within deadlines
- **Administrative Functions** - Application approvals, course management, reporting
- **Communication** - Automated notifications, announcements, in-app messaging

4.2.3 System Architecture Type



4.2.4 Technology Stack

Based on the requirements and Richfield's needs, we're using:

Frontend (Presentation Layer):

- **Vanilla JavaScript** - Clean, framework-free JavaScript (ES6+)
- **HTML5/CSS3** - Responsive design works on desktop, tablet, and mobile
- **21 Static Pages** - Login, dashboards, admin panels, student portal
- **Fetch API** - Native browser API for backend communication
- **Bootstrap 5** - UI components and responsive grid system

Purpose: The user interface that students, lecturers, and administrators interact with

Key Components:

- **Authentication Pages:** Login, Register, Password Reset
- **Student Portal:** Dashboard, Profile, Course Registration, Applications
- **Lecturer Portal:** Assigned Courses, Class Rosters, Announcements
- **Admin Portal:** Application Management, Course Management, Reports

Backend (Application Layer):

- **Node.js v20.x** - JavaScript runtime for server-side code
- **Express.js v5.2.1** - Web framework for building REST APIs
- **JWT (JSON Web Tokens)** - Secure authentication (7-day access, 30-day refresh tokens)
- **Bcrypt** - Password hashing with salt rounds
- **Nodemailer** - Email notifications (application status, registrations)
- **Sequelize ORM v6.37.5** - Database abstraction layer

Purpose: Handles business logic, data validation, and communicates with the database

Key Components:

- **Authentication Service:** Login, token generation, session management
- **User Management Service:** User accounts, roles, permissions
- **Application Service:** Application submission, approval workflow
- **Registration Service:** Course registration, prerequisite checking
- **Notification Service:** Email sending, in-app notifications
- **Reporting Service:** Generate reports and analytics

Database (Data Layer):

- **PostgreSQL 16** - Production-grade relational database
- **Sequelize ORM v6.37.5** - Database migrations, models, relationships
- **19+ Migrations** - Tracked schema evolution from initial setup
- **JSONB Fields** - Flexible document storage for complex data

Purpose: Stores all system data permanently

Key Components:

- **User Tables:** Users, Students, Emergency Contacts
- **Application Tables:** Applications, Application Documents
- **Course Tables:** Courses, Registrations
- **System Tables:** Audit Logs, Notifications, System Settings

File Storage:

- **Local file system** or **Cloud storage (AWS S3)** - For uploaded documents

Purpose: Stores uploaded documents (application documents, profile photos)

Approach: Files stored with unique identifiers, paths stored in database

Deployment & DevOps:

- **Git/GitHub** - Version control with collaborative workflows
- **Docker Compose** - Multi-container orchestration (app + database)
- **GitHub Actions** - Automated CI/CD pipeline
- **Let's Encrypt** - Automatic SSL certificate management
- **NGINX** - Reverse proxy and static file serving

Purpose: Stores upload We will focus on the deployment and serving of the entire system

Approach: Should be able to run the project on local machine (laptop / Desktop) and replicate it in production

4.2.5 Why We Chose This Stack for EduHub

Based on our team's skills assessment (from planning phase) and Richfield's requirements:

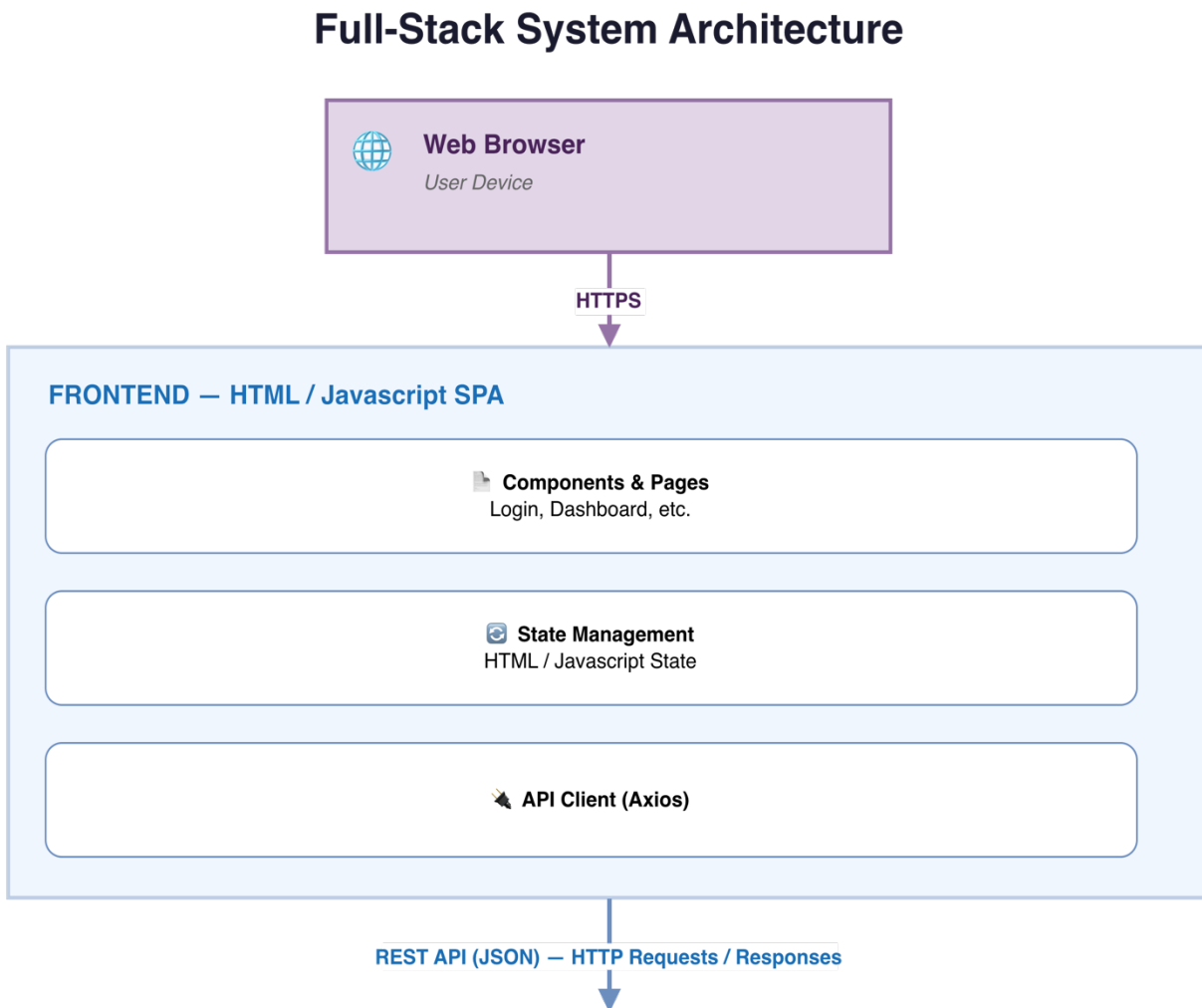
- **Team Already Knows JavaScript:** Our development team has JavaScript experience from coursework
- **Richfield's Growth Plans:** IT Manager mentioned growth from 1,200 to 2,000+ students - this stack can scale
- **Budget Constraints:** Richfield wants open-source solution (no licensing costs) - confirmed in workshops
- **Hosting Options:** Can deploy to free tier (Heroku/Railway) initially, then scale to AWS/DigitalOcean
- **Support Available:** Large community means we can find help when stuck

4.3 Architectural Design

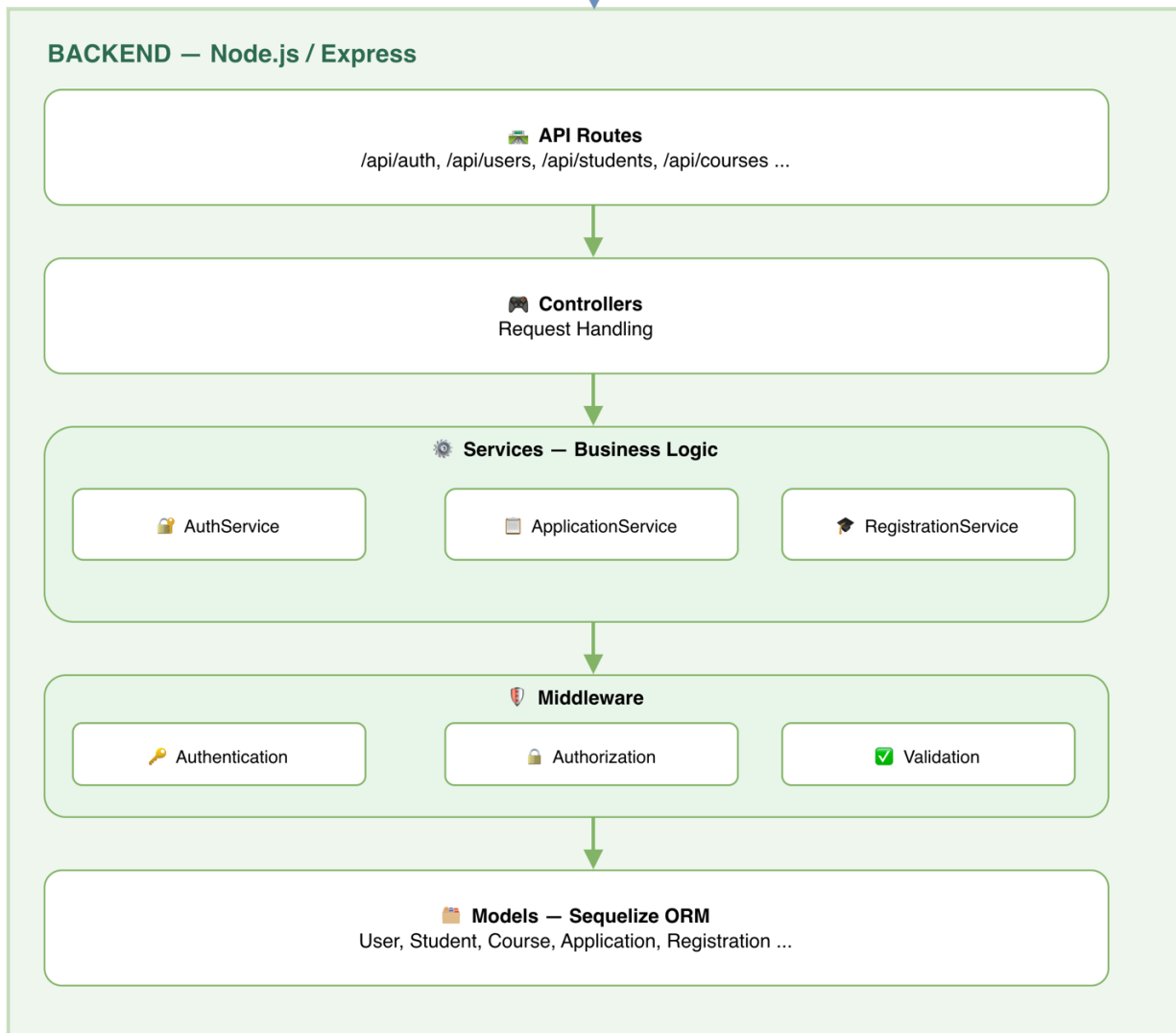
4.3.1 Software Architecture

We're using a **three-tier client-server architecture** with a **RESTful API** connecting the frontend and backend.

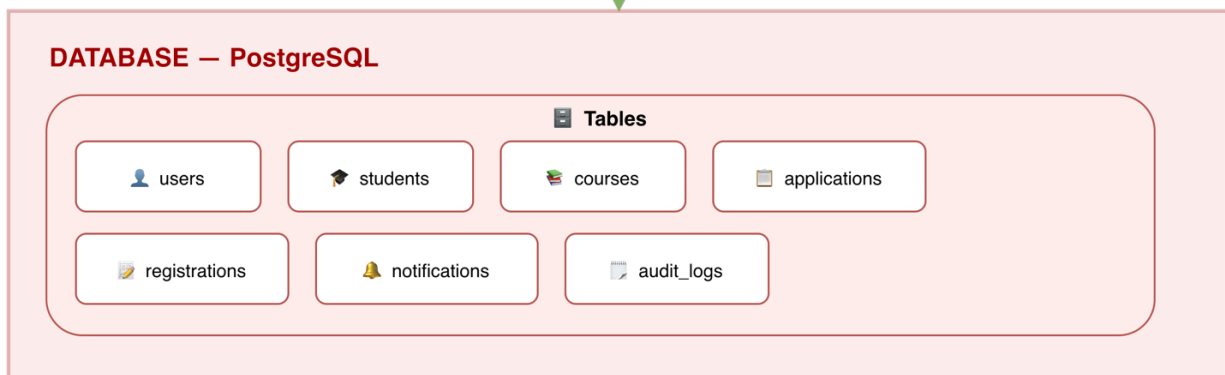
Architecture Diagram



REST API (JSON) — HTTP Requests / Responses



SQL Queries



Component Interaction Flow

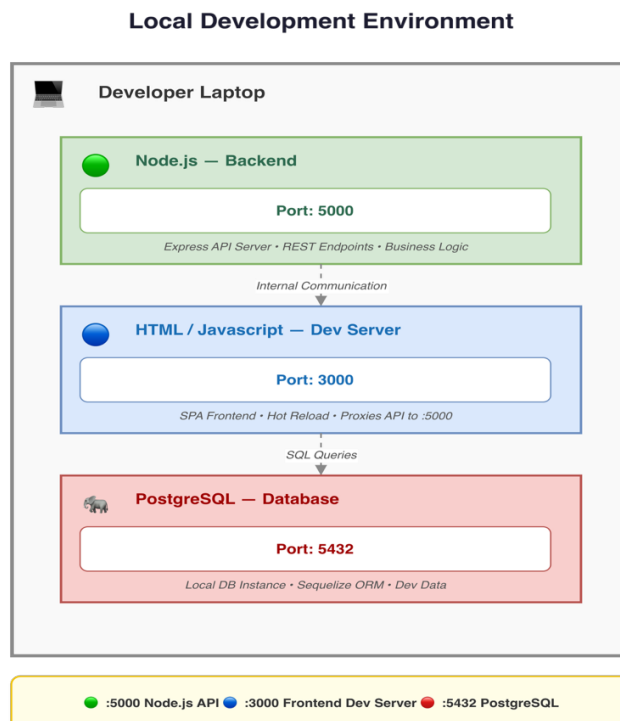
Example: Student Registers for a Course

- **Student clicks “Register” button** → Frontend (HTML/Javascript)
- **Frontend sends POST request** → /api/registrations with student_id and course_id
- **Backend receives request** → API Router directs to RegistrationController
- **Authentication middleware checks** → Is user logged in? Valid token?
- **Authorization middleware checks** → Is user a Student? Can they register?
- **Controller calls RegistrationService** → Business logic
- **Service checks prerequisites** → Query database for student’s completed courses
- **Service checks capacity** → Query database for current enrollment count
- **Service checks conflicts** → Query for schedule conflicts
- **Service creates registration** → Insert into registrations table
- **Service sends notification** → Queue email notification
- **Service returns success** → Back to Controller
- **Controller sends response** → JSON with registration details
- **Frontend updates UI** → Shows “Successfully registered!” message

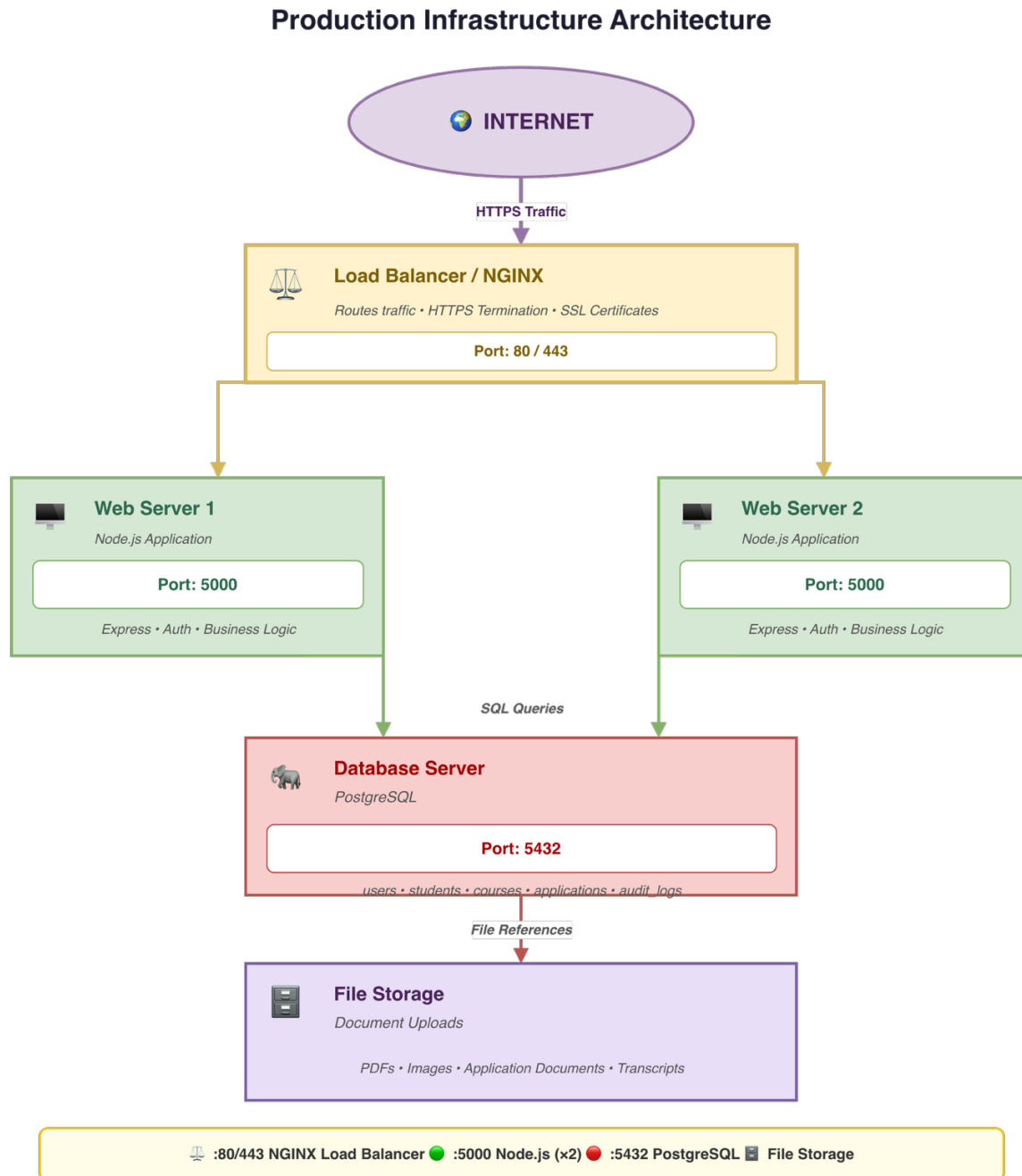
4.3.2 Hardware Architecture

Deployment Architecture

Development Environment (During Phase 5 - Implementation):



Production Environment (After deployment):



Hardware Requirements

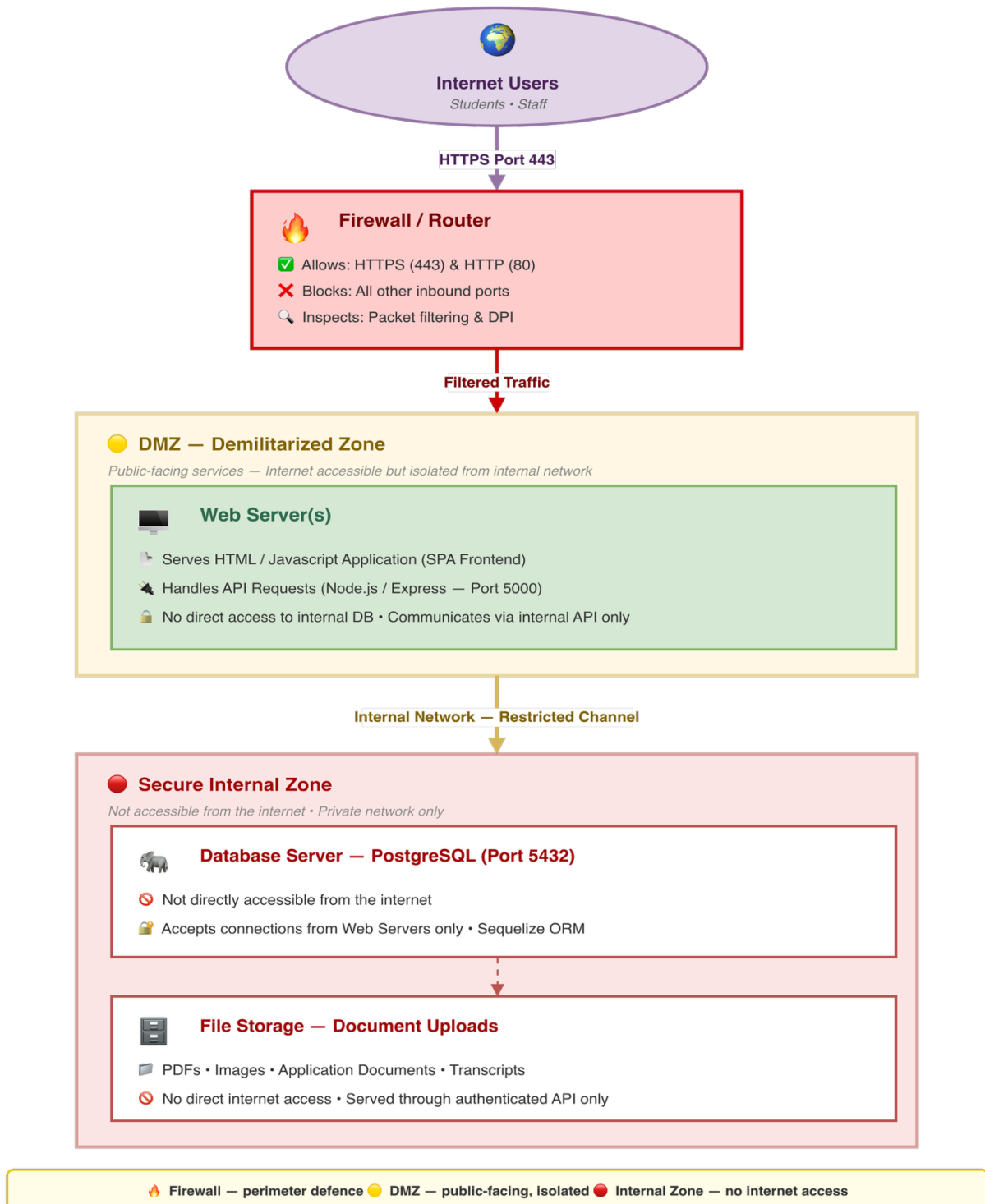
Minimum Server Specifications (For Production):

- **CPU:** 4 cores, 2.5 GHz
- **RAM:** 8 GB minimum, 16 GB recommended
- **Storage:** 100 GB SSD
- **Network:** 100 Mbps connection

4.3.3 Network Architecture

Network Topology

Network Security Architecture

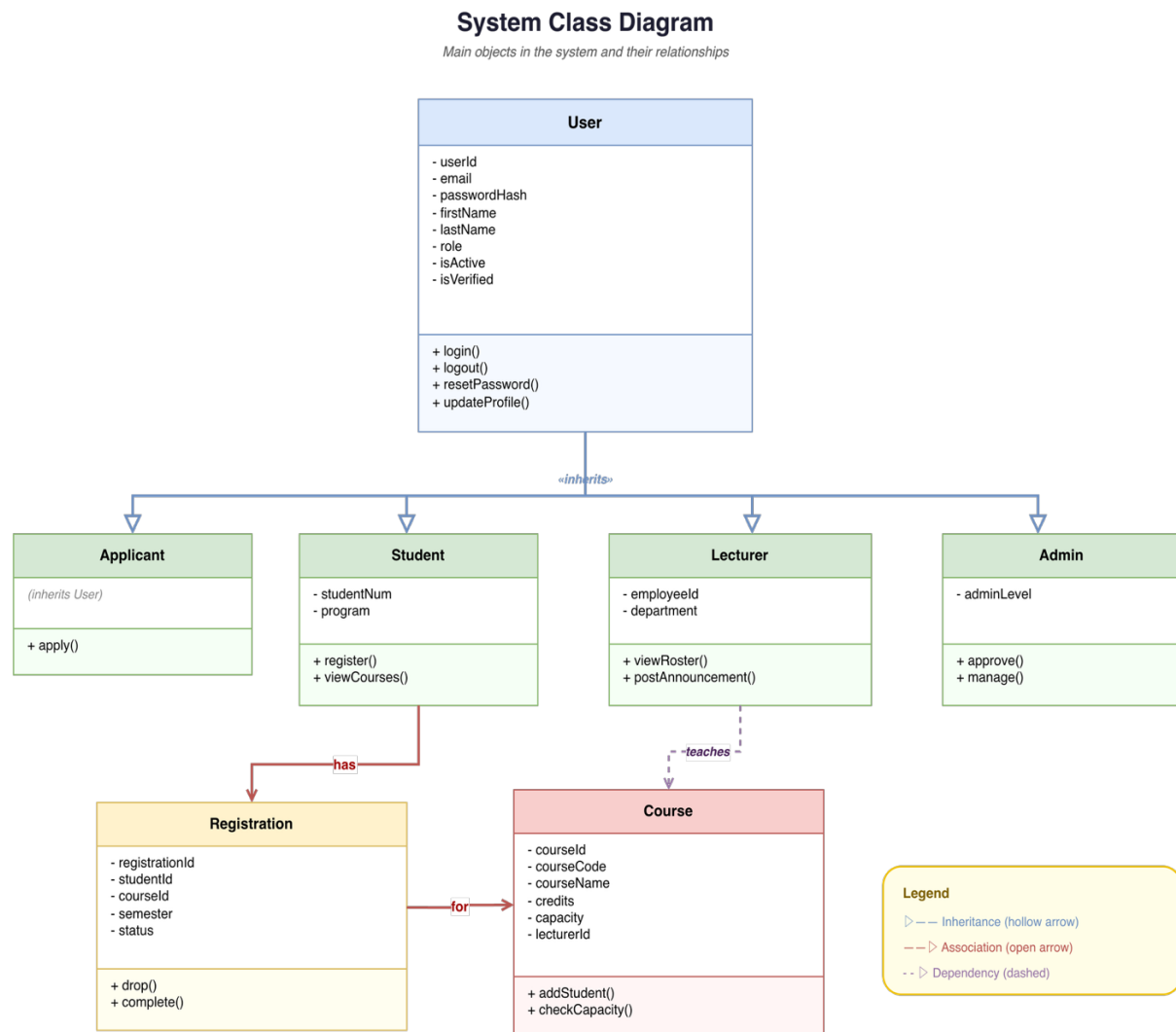


Network Security

- **Firewall:** Only allows HTTPS (port 443) and HTTP (port 80, redirects to HTTPS)
- **Database Isolation:** Database not directly accessible from internet
- **VPN Access:** Administrators access internal systems via VPN
- **SSL/TLS Encryption:** All data transmitted over HTTPS

4.3.4 Class Diagram

The class diagram shows the main objects in our system and how they relate to each other.



4.3.5 Actual Deployment Architecture

Docker Compose Setup

Production deployment uses **Docker Compose** with 3 services:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: eduhub
      POSTGRES_USER: eduhub_user
      POSTGRES_PASSWORD: [secure_password]
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgres://eduhub_user:***@postgres:5432/eduhub
      JWT_SECRET: [secure_secret]
      NODE_ENV: production
    depends_on:
      - postgres
    ports:
      - "5000:5000"

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./frontend:/usr/share/nginx/html
      - ./ssl:/etc/nginx/ssl
    depends_on:
      - backend
```

GitHub Actions CI/CD Pipeline

Automated deployment workflow (.github/workflows/deploy.yml):

Triggers: Push to main branch or manual trigger

Steps:

- **Build & Test** - Lint code, run tests
- **Build Docker Images** - Backend + Database
- **Deploy to Server** - SSH into production, pull images

- **Health Check** - Verify deployment success
- **Rollback** - Automatic rollback if health check fails

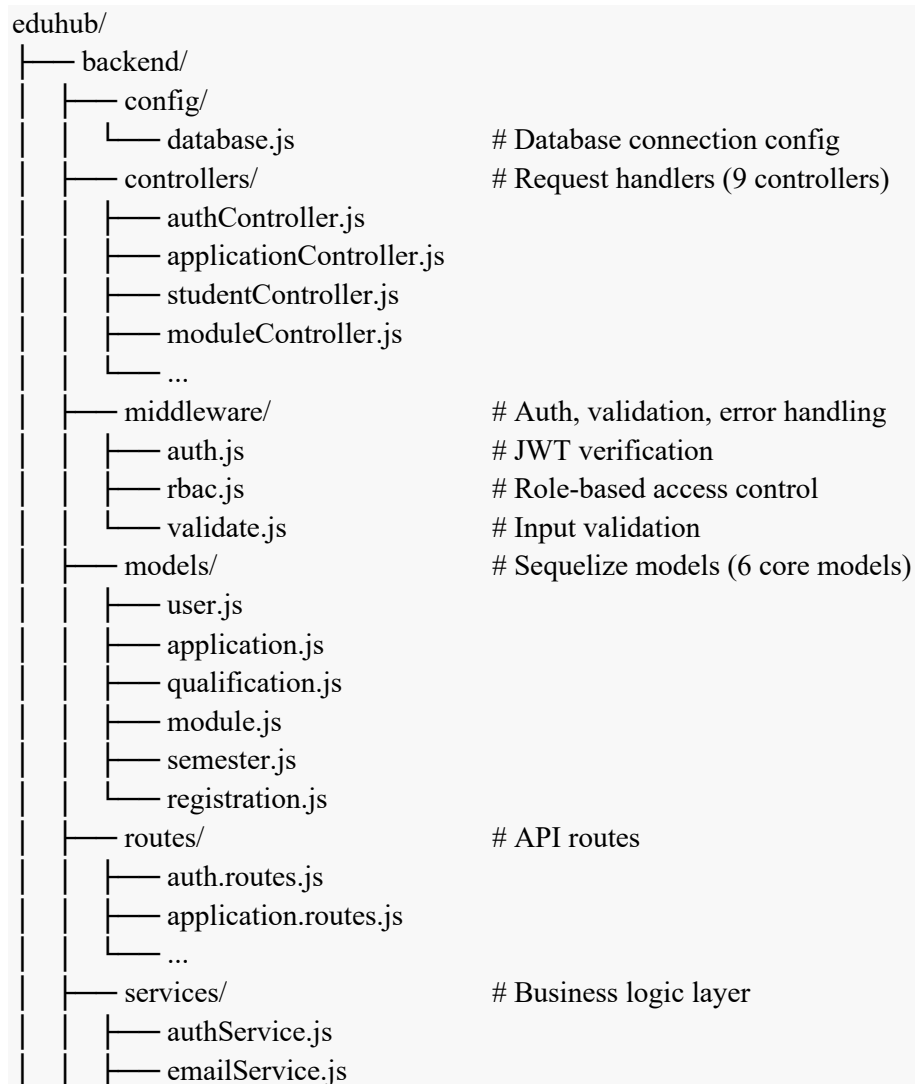
Deployment Time: ~5 minutes from commit to production

SSL/TLS Configuration

- **Let's Encrypt** - Free SSL certificates
- **Auto-renewal** - Certbot handles certificate renewal
- **HTTPS Redirect** - All HTTP traffic redirected to HTTPS
- **HSTS Header** - Strict-Transport-Security enabled

4.4 Physical Design

4.4.1 Actual Project Structure



```

├── ...
├── migrations/                                # Database migrations (19+)
│   ├── 001-create-users.js
│   ├── 002-create-applications.js
│   └── ...
├── utils/                                    # Helper functions
│   ├── generateStudentNumber.js
│   ├── luhnValidator.js
│   └── ...
├── server.js                                # Express app entry point
├── frontend/
│   ├── admin/                                # Admin portal pages
│   │   ├── dashboard.html
│   │   ├── applications.html
│   │   ├── manage-modules.html
│   │   └── ...
│   ├── student/                             # Student portal pages
│   │   ├── dashboard.html
│   │   ├── register-modules.html
│   │   ├── view-results.html
│   │   └── ...
│   ├── lecturer/                           # Lecturer portal pages
│   │   ├── dashboard.html
│   │   ├── view-classes.html
│   │   └── ...
│   ├── css/                                # Stylesheets
│   │   └── styles.css
│   ├── js/                                 # Frontend JavaScript
│   │   ├── api.js                          # API client
│   │   ├── auth.js                        # Auth helpers
│   │   └── utils.js
│   ├── login.html
│   └── index.html
├── .github/
│   └── workflows/
│       └── deploy.yml                       # CI/CD pipeline
├── docker-compose.yml                       # Multi-container setup
├── Dockerfile                             # Backend container config
├── .env.example                            # Environment variables template
└── package.json                           # Dependencies

```


Total Structure:

- 6 core database models
- 9 controllers
- 50+ API endpoints
- 21 frontend pages
- 19+ database migrations
- Automated CI/CD pipeline

4.4.2 Actual API Endpoints (50+)

Based on implementation analysis, EduHub has **50+ REST API endpoints**:

Authentication APIs (5 endpoints)

- POST /api/auth/register - Create new user account
- POST /api/auth/login - Authenticate and get JWT token
- POST /api/auth/refresh - Refresh access token
- POST /api/auth/logout - Invalidate session
- POST /api/auth/reset-password - Password recovery

Application APIs (6 endpoints)

- GET /api/applications - List all applications (paginated)
- POST /api/applications - Submit new application
- GET /api/applications/:id - Get application details
- PUT /api/applications/:id/status - Approve/reject application (admin)
- POST /api/applications/:id/documents - Upload supporting documents
- DELETE /api/applications/:id - Withdraw application

Student APIs (5 endpoints)

- GET /api/students - List students (admin only)
- GET /api/students/:id - Get student profile
- PUT /api/students/:id - Update student details
- GET /api/students/:id/registrations - Student's registered modules
- GET /api/students/:id/transcript - Academic transcript

Module APIs (8 endpoints)

- GET /api/modules - List all available modules
- GET /api/modules/:id - Module details
- POST /api/modules - Create module (admin)
- PUT /api/modules/:id - Update module (admin)
- DELETE /api/modules/:id - Remove module (admin)
- GET /api/modules/:id/students - Enrolled students

- GET /api/modules/search - Search modules by criteria
- GET /api/qualifications/:id/modules - Modules in a qualification

Registration APIs (5 endpoints)

- POST /api/registrations - Register for module(s)
- GET /api/registrations/:id - Registration details
- DELETE /api/registrations/:id - Drop module
- GET /api/registrations/semester/:id - Registrations per semester
- PUT /api/registrations/:id/grade - Record final grade (lecturer)

Admin APIs (4 endpoints)

- GET /api/admin/dashboard - System statistics
- GET /api/admin/users - User management
- PUT /api/admin/users/:id/role - Change user roles
- GET /api/admin/reports - Generate system reports

Reference Data APIs (5 endpoints)

- GET /api/qualifications - List all programs
- GET /api/qualifications/:id - Program details
- GET /api/semesters - Academic periods
- GET /api/semesters/current - Current active semester
- GET /api/departments - Department list

Utility APIs (2 endpoints)

- GET /api/health - System health check
- GET /api/version - API version info

Total: 50+ endpoints organized in RESTful resource-based architecture.

4.4.2 System Deployment Model

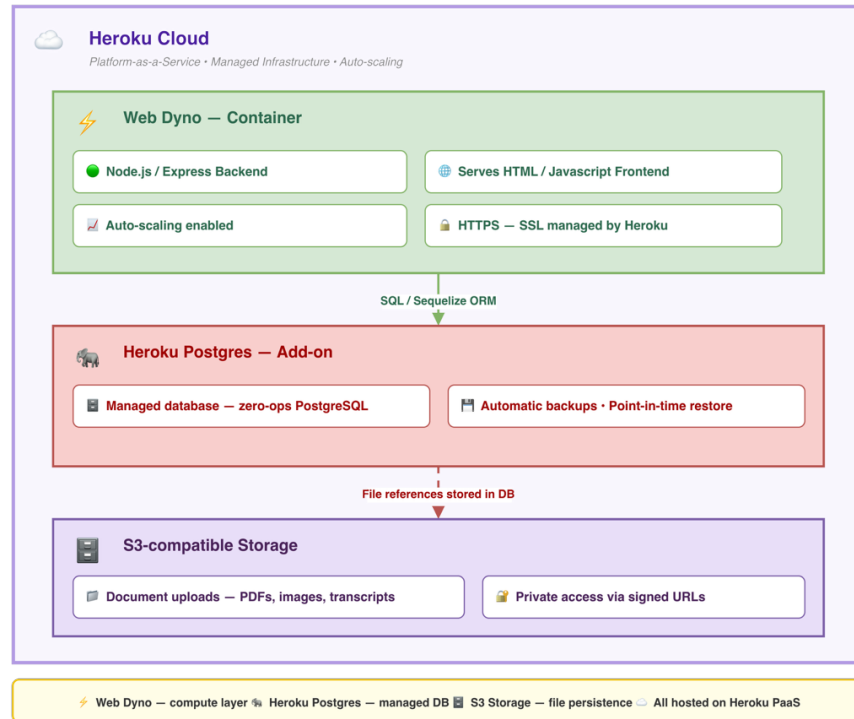
The physical design describes how the logical components (software) are deployed onto physical hardware and infrastructure.

Deployment Options

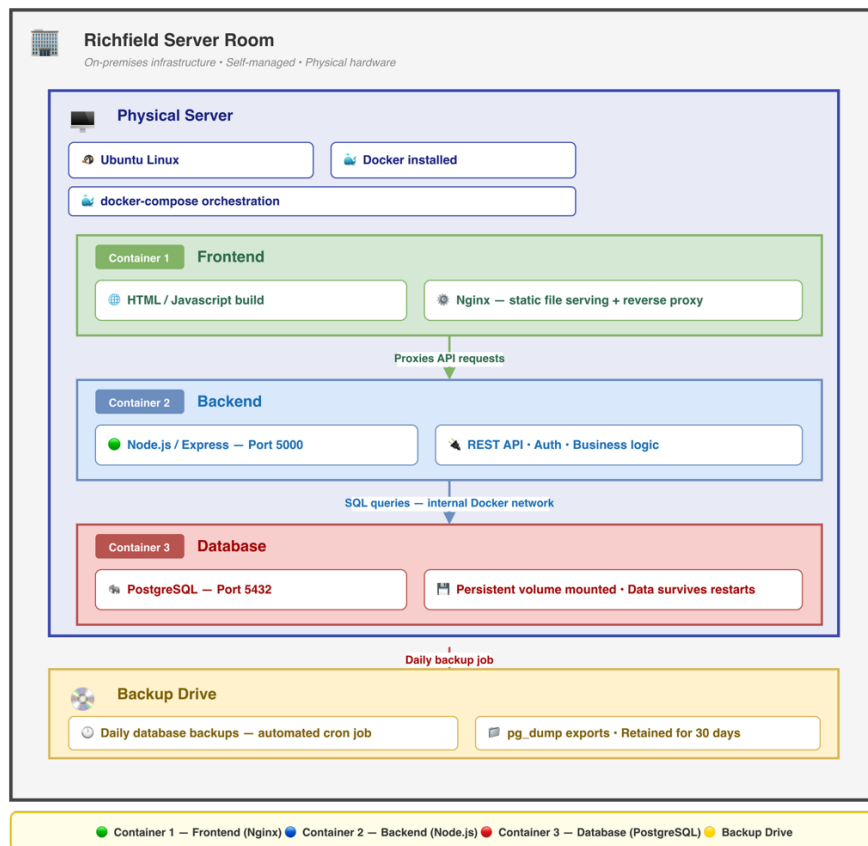
Option 1: Cloud Deployment (Recommended for Richfield)

Why Cloud?

- No need to buy/maintain physical servers
- Scales automatically as Richfield grows
- Provider handles backups, security patches, infrastructure
- Pay only for what you use



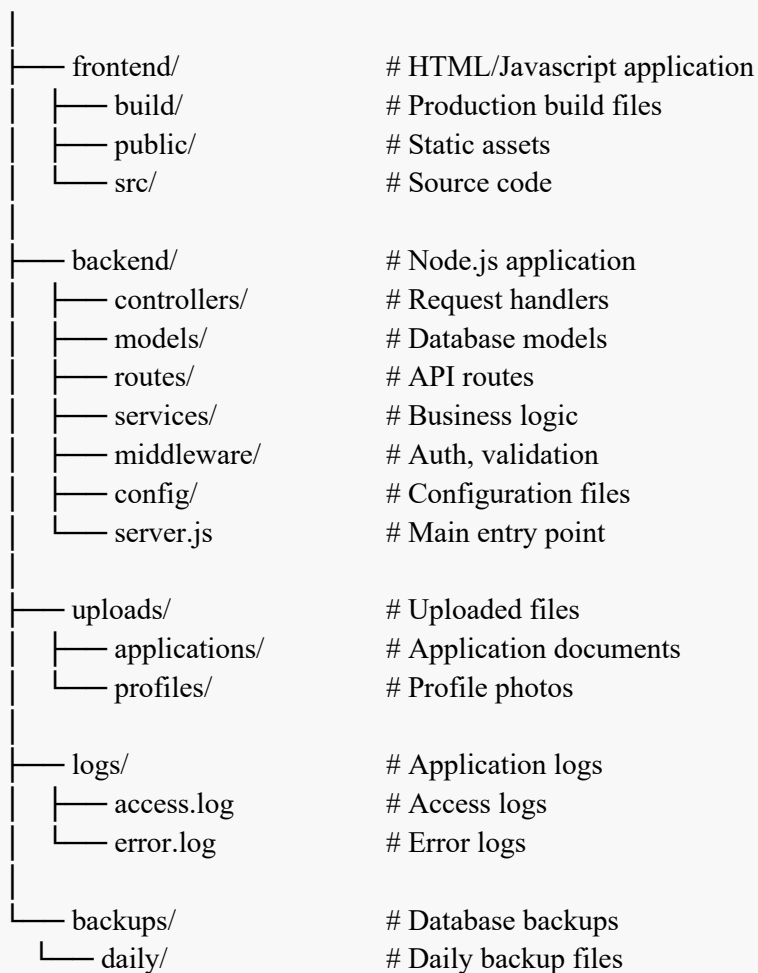
Option 2: On-Premises Server (if Richfield prefers / Used for demo)



4.4.3 File Organization Structure

On the Server:

/var/www/eduhub/



4.4.4 Capacity Planning

Based on Richfield's expected usage:

Current State:

- Estimated 500-1000 students
- 50 lecturers
- 10 administrative staff

Storage Needs:

- Database: ~5 GB initially
- Uploaded documents: ~20 GB initially
- Total: ~30 GB (with room to grow to 500 GB)

Bandwidth Needs:

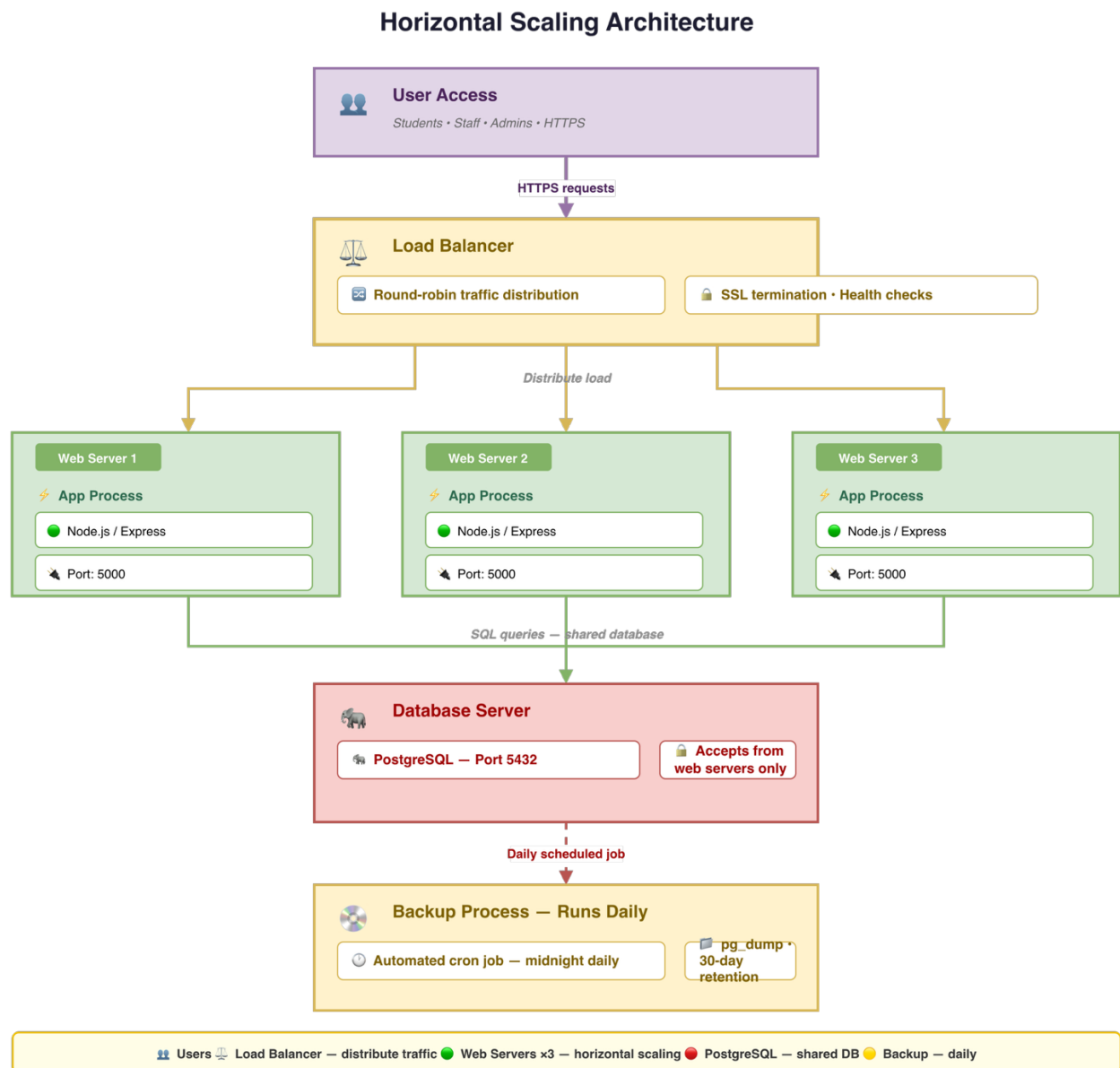
- Average concurrent users: 50-100
- Peak concurrent users (registration period): 200-500
- Bandwidth: 10-50 Mbps

Processing Needs:

- Regular load: 2-4 CPU cores
- Peak load (registration): 4-8 CPU cores with auto-scaling

4.4.5 Process Flow Diagram

Shows how different system processes interact:



4.5 Database Design

Now we get into the detailed database design - the actual table structures with specific data types, constraints, and relationships that will be implemented in PostgreSQL.

4.5.1 Database Schema Overview

The database consists of **10 primary tables** organized into logical groups:

- **Authentication & Users:** Users
- **Student Information:** Students, Emergency_Contacts
- **Applications:** Applications, Application_Documents
- **Courses:** Courses, Registrations
- **System:** System_Settings, Audit_Logs, Notifications

4.5.2 Complete Database Schema

Users Table

Purpose: Stores authentication and basic information for all system users

SQL Schema:

```
CREATE TABLE users (  
  user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  role VARCHAR(20) NOT NULL CHECK (role IN ('Applicant', 'Student', 'Lecturer', 'Admin', 'Alumni')),  
  is_active BOOLEAN DEFAULT TRUE,  
  is_verified BOOLEAN DEFAULT FALSE,  
  mfa_enabled BOOLEAN DEFAULT FALSE,  
  mfa_secret VARCHAR(255),  
  last_login TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_users_email ON users(email);  
CREATE INDEX idx_users_role ON users(role);  
CREATE INDEX idx_users_is_active ON users(is_active);
```

Column	Type	Constraints	Description
user_id	UUID	PRIMARY KEY	Unique identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address (used for login)
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password (never store plain passwords!)
first_name	VARCHAR(100)	NOT NULL	User's first name
last_name	VARCHAR(100)	NOT NULL	User's last name
role	VARCHAR(20)	NOT NULL, CHECK	User role (Applicant, Student, Lecturer, Admin, Alumni)
is_active	BOOLEAN	DEFAULT TRUE	Whether account is active
is_verified	BOOLEAN	DEFAULT FALSE	Whether email has been verified
mfa_enabled	BOOLEAN	DEFAULT FALSE	Whether multi-factor authentication is enabled
mfa_secret	VARCHAR(255)	NULLABLE	Secret key for MFA (if enabled)
last_login	TIMESTAMP	NULLABLE	Last login timestamp
created_at	TIMESTAMP	DEFAULT NOW()	When account was created
updated_at	TIMESTAMP	DEFAULT NOW()	Last update to record

Relationships:

- One-to-One with Students (when role = 'Student')
- One-to-Many with Applications
- One-to-Many with Audit_Logs
- One-to-Many with Notifications

Students Table

Purpose: Stores additional information specific to enrolled students

SQL Schema:

```
CREATE TABLE students (
  student_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID UNIQUE NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  student_number VARCHAR(20) UNIQUE NOT NULL,
  id_number VARCHAR(50) UNIQUE NOT NULL,
  phone VARCHAR(20) NOT NULL,
  address TEXT NOT NULL,
  date_of_birth DATE NOT NULL,
  program VARCHAR(100) NOT NULL,
  year_of_study INTEGER DEFAULT 1 CHECK (year_of_study BETWEEN 1 AND 6),
  status VARCHAR(20) DEFAULT 'Active' CHECK (status IN ('Active', 'Graduated', 'Suspended')),
  profile_photo_url VARCHAR(255),
  enrollment_date DATE NOT NULL,
```

```

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE INDEX idx_students_student_number ON students(student_number);
CREATE INDEX idx_students_user_id ON students(user_id);
CREATE INDEX idx_students_id_number ON students(id_number);
CREATE INDEX idx_students_status ON students(status);

```

Column	Type	Constraints	Description
student_id	UUID	PRIMARY KEY	Unique identifier
user_id	UUID	FOREIGN KEY, UNIQUE	Reference to users table
student_number	VARCHAR(20)	UNIQUE, NOT NULL	Generated student number (format: YEAR-####)
id_number	VARCHAR(50)	UNIQUE, NOT NULL	National ID number
phone	VARCHAR(20)	NOT NULL	Student phone number
address	TEXT	NOT NULL	Physical address
date_of_birth	DATE	NOT NULL	Date of birth
program	VARCHAR(100)	NOT NULL	Academic program enrolled in
year_of_study	INTEGER	CHECK 1-6	Current year of study
status	VARCHAR(20)	CHECK	Student status (Active, Graduated, Suspended)
profile_photo_url	VARCHAR(255)	NULLABLE	URL/path to profile photo
enrollment_date	DATE	NOT NULL	When student enrolled
created_at	TIMESTAMP	DEFAULT NOW()	Record creation date
updated_at	TIMESTAMP	DEFAULT NOW()	Last update date

Relationships:

- One-to-One with Users
- One-to-Many with Emergency_Contacts
- Many-to-Many with Courses (through Registrations)

Emergency_Contacts Table

Purpose: Stores emergency contact information for students (max 3 per student)

SQL Schema:

```
CREATE TABLE emergency_contacts (  
  contact_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  student_id UUID NOT NULL REFERENCES students(student_id) ON DELETE CASCADE,  
  name VARCHAR(100) NOT NULL,  
  relationship VARCHAR(50) NOT NULL,  
  phone VARCHAR(20) NOT NULL,  
  email VARCHAR(255),  
  is_primary BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_emergency_contacts_student_id ON emergency_contacts(student_id);
```

Column	Type	Constraints	Description
contactid	UUID	PRIMARY KEY	Unique identifier
studentid	UUID	FOREIGN KEY	Reference to students table
name	VARCHAR(100)	NOT NULL	Contact's full name
relationship	VARCHAR(50)	NOT NULL	Relationship to student
phone	VARCHAR(20)	NOT NULL	Contact phone number
email	VARCHAR(255)	NULLABLE	Contact email (optional)
isprimary	BOOLEAN	DEFAULT FALSE	Whether this is the primary contact
createdat	TIMESTAMP	DEFAULT NOW()	Record creation date
updatedat	TIMESTAMP	DEFAULT NOW()	Last update date

Business Rule: Maximum 3 emergency contacts per student (enforced in application logic)

Applications Table

Purpose: Stores student applications for admission

SQL Schema:

```
CREATE TABLE applications (  
  application_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  id_number VARCHAR(50) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  phone VARCHAR(20) NOT NULL,
```

```

address TEXT NOT NULL,
date_of_birth DATE NOT NULL,
program VARCHAR(100) NOT NULL,
status VARCHAR(20) DEFAULT 'Submitted' CHECK (status IN ('Submitted', 'Under Review', 'Approved', 'Rejected', 'Withdrawn')),
rejection_reason TEXT,
reviewed_by UUID REFERENCES users(user_id) ON DELETE SET NULL,
reviewed_at TIMESTAMP,
is_draft BOOLEAN DEFAULT TRUE,
submitted_at TIMESTAMP,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_applications_user_id ON applications(user_id);
CREATE INDEX idx_applications_status ON applications(status);
CREATE INDEX idx_applications_submitted_at ON applications(submitted_at);

```

Application Documents Table

Purpose: Stores documents uploaded with applications (ID, certificates, transcripts)

SQL Schema:

```

CREATE TABLE application_documents (
    document_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    application_id UUID NOT NULL REFERENCES applications(application_id) ON DELETE CASCADE,
    document_type VARCHAR(50) NOT NULL CHECK (document_type IN ('ID', 'Certificate', 'Transcript', 'Other')),
    file_name VARCHAR(255) NOT NULL,
    file_path VARCHAR(255) NOT NULL,
    file_size INTEGER NOT NULL CHECK (file_size <= 5242880), -- 5MB max
    mime_type VARCHAR(100) NOT NULL,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_application_documents_application_id ON application_documents(application_id);

```

Column	Type	Constraints	Description
document_id	UUID	PRIMARY KEY	Unique identifier
application_id	UUID	FOREIGN KEY	Reference to applications table
document_type	VARCHAR(50)	CHECK	Type of document

file_name	VARCHAR(255)	NOT NULL	Original filename
file_path	VARCHAR(255)	NOT NULL	Storage path
file_size	INTEGER	CHECK <= 5MB	File size in bytes
mime_type	VARCHAR(100)	NOT NULL	File MIME type (e.g., application/pdf)
uploaded_at	TIMESTAMP	DEFAULT NOW()	Upload timestamp

Courses Table

Purpose: Stores course information

SQL Schema:

```
CREATE TABLE courses (  
  course_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  course_code VARCHAR(20) UNIQUE NOT NULL,  
  course_name VARCHAR(200) NOT NULL,  
  description TEXT,  
  credits INTEGER NOT NULL CHECK (credits > 0),  
  capacity INTEGER NOT NULL CHECK (capacity > 0),  
  department VARCHAR(100) NOT NULL,  
  semester VARCHAR(50) NOT NULL,  
  schedule VARCHAR(100),  
  prerequisites TEXT,  
  lecturer_id UUID REFERENCES users(user_id) ON DELETE SET NULL,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_courses_course_code ON courses(course_code);  
CREATE INDEX idx_courses_lecturer_id ON courses(lecturer_id);  
CREATE INDEX idx_courses_semester ON courses(semester);  
CREATE INDEX idx_courses_is_active ON courses(is_active);
```

Column	Type	Constraints	Description
course_id	UUID	PRIMARY KEY	Unique identifier
course_code	VARCHAR(20)	UNIQUE, NOT NULL	Course code (e.g., CS101)
course_name	VARCHAR(200)	NOT NULL	Course name
description	TEXT	NULLABLE	Course description
credits	INTEGER	CHECK > 0	Credit hours

capacity	INTEGER	CHECK > 0	Maximum number of students
department	VARCHAR(100)	NOT NULL	Department offering course
semester	VARCHAR(50)	NOT NULL	Semester (e.g., Fall 2026)
schedule	VARCHAR(100)	NULLABLE	Class schedule
prerequisites	TEXT	NULLABLE	Prerequisite courses (JSON or comma-separated)
lecturer_id	UUID	FOREIGN KEY	Assigned lecturer
is_active	BOOLEAN	DEFAULT TRUE	Whether course is active
created_at	TIMESTAMP	DEFAULT NOW()	Record creation date
updated_at	TIMESTAMP	DEFAULT NOW()	Last update date

Registrations Table

Purpose: Junction table for student-course registrations (many-to-many relationship)

SQL Schema:

```
CREATE TABLE registrations (
  registration_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  student_id UUID NOT NULL REFERENCES students(student_id) ON DELETE RESTRICT,
  course_id UUID NOT NULL REFERENCES courses(course_id) ON DELETE RESTRICT,
  semester VARCHAR(50) NOT NULL,
  registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(20) DEFAULT 'Registered' CHECK (status IN ('Registered', 'Dropped', 'Completed')),
  dropped_at TIMESTAMP,
  grade VARCHAR(5),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(student_id, course_id, semester) -- Student can't register for same course twice in same semester
);
```

```
CREATE INDEX idx_registrations_student_id ON registrations(student_id);
```

```
CREATE INDEX idx_registrations_course_id ON registrations(course_id);
```

```
CREATE INDEX idx_registrations_semester ON registrations(semester);
```

```
CREATE INDEX idx_registrations_status ON registrations(status);
```

Column	Type	Constraints	Description
registration_id	UUID	PRIMARY KEY	Unique identifier
student_id	UUID	FOREIGN KEY	Reference to students table
course_id	UUID	FOREIGN KEY	Reference to courses table
semester	VARCHAR(50)	NOT NULL	Registration semester
registration_date	TIMESTAMP	DEFAULT NOW()	When student registered
status	VARCHAR(20)	CHECK	Registration status
dropped_at	TIMESTAMP	NULLABLE	When course was dropped

grade	VARCHAR(5)	NULLABLE	Final grade (future feature)
created_at	TIMESTAMP	DEFAULT NOW()	Record creation date
updated_at	TIMESTAMP	DEFAULT NOW()	Last update date

Unique Constraint: (student_id, course_id, semester) - Prevents duplicate registrations

System Settings Table

Purpose: Stores system configuration settings (registration periods, etc.)

SQL Schema:

```
CREATE TABLE system_settings (
  setting_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  setting_key VARCHAR(100) UNIQUE NOT NULL,
  setting_value TEXT NOT NULL,
  description TEXT,
  updated_by UUID REFERENCES users(user_id) ON DELETE SET NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_system_settings_setting_key ON system_settings(setting_key);
```

Column	Type	Constraints	Description
setting_id	UUID	PRIMARY KEY	Unique identifier
setting_key	VARCHAR(100)	UNIQUE, NOT NULL	Configuration key name
setting_value	TEXT	NOT NULL	Stored configuration value
description	TEXT	NULLABLE	Description of what the setting controls
updated_by	UUID	FOREIGN KEY	User who last updated the setting
updated_at	TIMESTAMP	DEFAULT NOW()	Last update timestamp

Common Settings:

- registration_start_date: When registration opens
- registration_end_date: When registration closes
- add_drop_deadline: Last day to add/drop courses
- current_semester: Current active semester
- max_credits_per_semester: Maximum credits students can register for

Audit Logs Table

Purpose: Tracks important user actions for security and compliance

SQL Schema:

```
CREATE TABLE audit_logs (  
  log_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID REFERENCES users(user_id) ON DELETE SET NULL,  
  action VARCHAR(100) NOT NULL,  
  entity_type VARCHAR(50) NOT NULL,  
  entity_id UUID,  
  changes JSONB,  
  ip_address VARCHAR(45),  
  user_agent VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX idx_audit_logs_user_id ON audit_logs(user_id);  
CREATE INDEX idx_audit_logs_entity_type ON audit_logs(entity_type);  
CREATE INDEX idx_audit_logs_entity_id ON audit_logs(entity_id);  
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at);
```

Column	Type	Constraints	Description
log_id	UUID	PRIMARY KEY	Unique identifier
user_id	UUID	FOREIGN KEY	User who performed action
action	VARCHAR(100)	NOT NULL	Action performed (e.g., USER_LOGIN)
entity_type	VARCHAR(50)	NULLABLE	Type of entity affected
entity_id	UUID	NULLABLE	Identifier of entity affected
details	TEXT	NULLABLE	Additional metadata/details
ip_address	VARCHAR(45)	NULLABLE	Client IP address
user_agent	TEXT	NULLABLE	Browser/device information
created_at	TIMESTAMP	DEFAULT NOW()	When action occurred

Notifications Table

Purpose: Stores in-app notifications for users

SQL Schema:

```
CREATE TABLE notifications (  
  notification_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  title VARCHAR(200) NOT NULL,  
  message TEXT NOT NULL,  
  type VARCHAR(50) NOT NULL CHECK (type IN ('info', 'success', 'warning', 'error')),
```

```

is_read BOOLEAN DEFAULT FALSE,
link_url VARCHAR(255),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_notifications_is_read ON notifications(is_read);
CREATE INDEX idx_notifications_created_at ON notifications(created_at);

```

Column	Type	Constraints	Description
notification_id	UUID	PRIMARY KEY	Unique identifier
user_id	UUID	FOREIGN KEY	Recipient user
title	VARCHAR(200)	NOT NULL	Notification title
message	TEXT	NOT NULL	Notification body
type	VARCHAR(20)	CHECK	Notification type (info, success, warning, error)
is_read	BOOLEAN	DEFAULT FALSE	Whether notification has been read
action_url	VARCHAR(255)	NULLABLE	Related link or page path
created_at	TIMESTAMP	DEFAULT NOW()	When notification was created
read_at	TIMESTAMP	NULLABLE	When notification was read

4.5.3 Database Relationships Summary

One-to-One Relationships

- **Users ↔ Students:** Each student record belongs to one user account

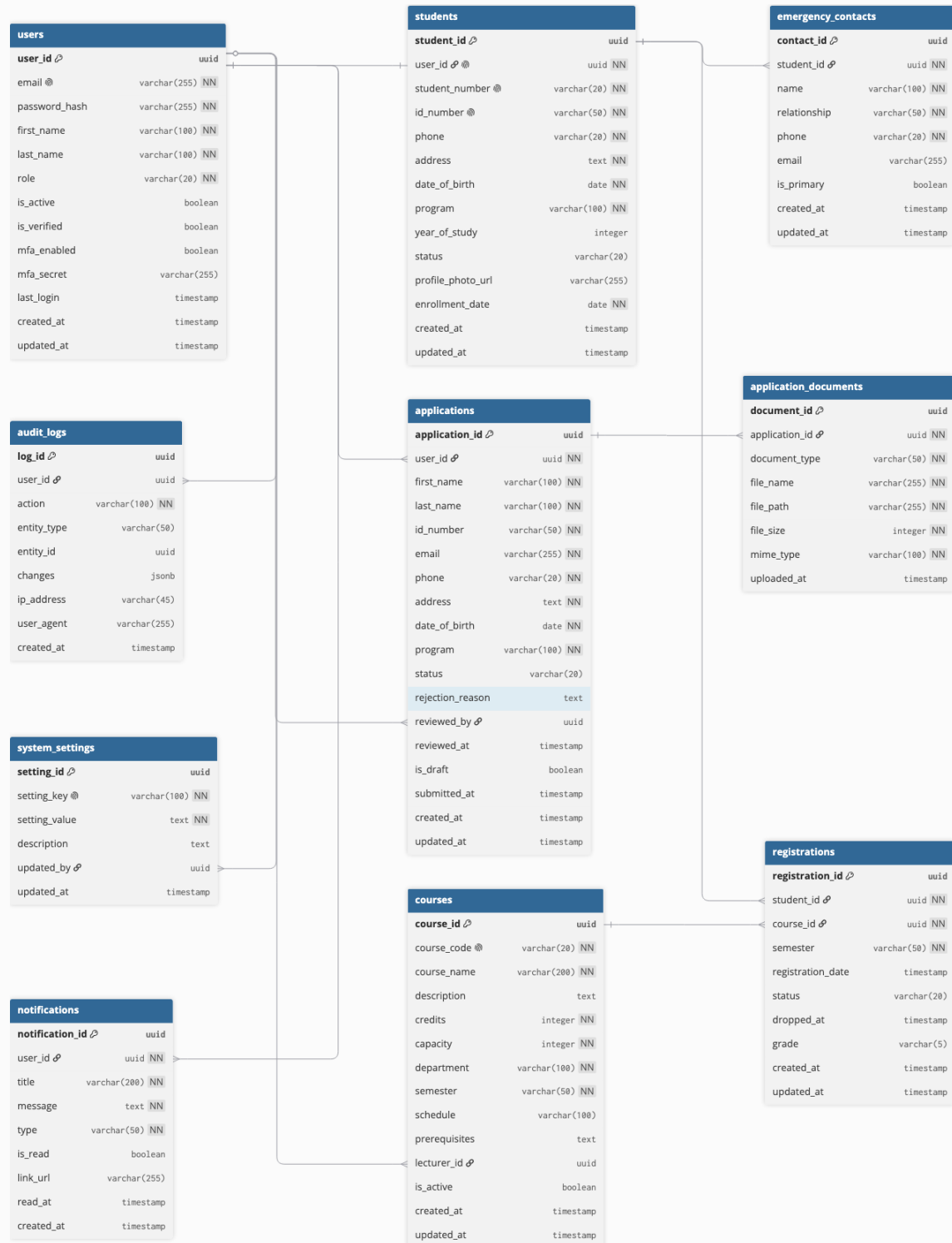
One-to-Many Relationships

- **Users → Applications** (as applicant): One user can submit multiple applications
- **Users → Applications** (as reviewer): One admin can review multiple applications
- **Users → Courses:** One lecturer can teach multiple courses
- **Users → Audit_Logs:** One user performs many actions
- **Users → Notifications:** One user receives many notifications
- **Students → Emergency_Contacts:** One student has multiple emergency contacts
- **Applications → Application_Documents:** One application has multiple documents
- **Courses → Registrations:** One course has multiple student registrations

Many-to-Many Relationships

- **Students ↔ Courses** (through Registrations): Students can register for multiple courses, courses have multiple students

4.5.4 Database Diagram (ERD):



4.6 Program Design (Pseudocode)

This section shows the logic for key system processes using pseudocode - a human-readable description of how the code will work, without getting into specific programming syntax.

4.6.1 User Authentication - Login Process

```
FUNCTION loginUser(email, password):
    // Step 1: Validate input
    IF email is empty OR password is empty THEN
        RETURN error "Email and password are required"
    END IF

    IF email is not valid email format THEN
        RETURN error "Invalid email format"
    END IF

    // Step 2: Find user in database
    user = DATABASE.findOne("users", WHERE email = email)

    IF user is null THEN
        LOG_EVENT("Failed login attempt - user not found", email)
        RETURN error "Invalid email or password"
    END IF

    // Step 3: Check if account is active
    IF user.is_active = FALSE THEN
        RETURN error "Account is disabled. Please contact support."
    END IF

    // Step 4: Verify password
    passwordMatches = BCrypt.compare(password, user.password_hash)

    IF passwordMatches = FALSE THEN
        LOG_EVENT("Failed login attempt - wrong password", email)
        // Increment failed attempts counter
        INCREMENT user.failed_login_attempts
        IF user.failed_login_attempts >= 5 THEN
            user.is_active = FALSE
            DATABASE.update("users", user)
            RETURN error "Account locked due to multiple failed attempts"
        END IF
        RETURN error "Invalid email or password"
    END IF
```

```

// Step 5: Check MFA if enabled
IF user.mfa_enabled = TRUE THEN
    // Prompt for MFA code (handled separately)
    RETURN "MFA_REQUIRED"
END IF

// Step 6: Generate JWT token
tokenPayload = {
    userId: user.user_id,
    email: user.email,
    role: user.role
}
token = JWT.sign(tokenPayload, SECRET_KEY, expiresIn="24h")

// Step 7: Update last login
user.last_login = CURRENT_TIMESTAMP()
user.failed_login_attempts = 0
DATABASE.update("users", user)

// Step 8: Log successful login
LOG_AUDIT("USER_LOGIN", user.user_id, IP_ADDRESS, USER_AGENT)

// Step 9: Return success with token
RETURN {
    success: true,
    token: token,
    user: {
        userId: user.user_id,
        email: user.email,
        firstName: user.first_name,
        lastName: user.last_name,
        role: user.role
    }
}
END FUNCTION

```

4.6.2 Application Submission Process

```
FUNCTION submitApplication(applicationData, userId):
    // Step 1: Validate user is logged in and is an Applicant
    user = DATABASE.findOne("users", WHERE user_id = userId)

    IF user.role != "Applicant" THEN
        RETURN error "Only applicants can submit applications"
    END IF

    // Step 2: Check if user already has a pending/approved application
    existingApp = DATABASE.findOne("applications",
        WHERE user_id = userId AND status IN ("Submitted", "Under Review", "Approved"))

    IF existingApp exists THEN
        RETURN error "You already have a pending or approved application"
    END IF

    // Step 3: Validate application data
    requiredFields = ["first_name", "last_name", "id_number", "email", "phone",
        "address", "date_of_birth", "program"]

    FOR EACH field IN requiredFields:
        IF applicationData[field] is empty THEN
            RETURN error "Required field missing: " + field
        END IF
    END FOR

    // Validate ID number format
    IF NOT MATCHES_PATTERN(applicationData.id_number, "^\d{13}$") THEN
        RETURN error "ID number must be 13 digits"
    END IF

    // Validate email format
    IF NOT IS_VALID_EMAIL(applicationData.email) THEN
        RETURN error "Invalid email format"
    END IF

    // Validate age (must be at least 16)
    age = CALCULATE_AGE(applicationData.date_of_birth)
    IF age < 16 THEN
        RETURN error "Applicant must be at least 16 years old"
    END IF
```

```

// Step 4: Create application record
application = {
  application_id: GENERATE_UUID(),
  user_id: userId,
  first_name: applicationData.first_name,
  last_name: applicationData.last_name,
  id_number: applicationData.id_number,
  email: applicationData.email,
  phone: applicationData.phone,
  address: applicationData.address,
  date_of_birth: applicationData.date_of_birth,
  program: applicationData.program,
  status: "Submitted",
  is_draft: FALSE,
  submitted_at: CURRENT_TIMESTAMP(),
  created_at: CURRENT_TIMESTAMP()
}

DATABASE.insert("applications", application)

// Step 5: Log the action
LOG_AUDIT("APPLICATION_SUBMITTED", userId, "Application", application.application_id)

// Step 6: Send confirmation email
SEND_EMAIL({
  to: application.email,
  subject: "Application Received",
  body: "Your application for " + application.program + " has been received.
        Application ID: " + application.application_id
})

// Step 7: Send in-app notification
CREATE_NOTIFICATION(userId, "Application Submitted",
  "Your application has been submitted successfully!", "success")

RETURN {
  success: true,
  applicationId: application.application_id,
  message: "Application submitted successfully"
}
END FUNCTION

```

4.6.3 Application Approval Process

```
FUNCTION approveApplication(applicationId, adminUserId):
    // Step 1: Verify admin permissions
    admin = DATABASE.findOne("users", WHERE user_id = adminUserId)

    IF admin.role != "Admin" THEN
        RETURN error "Only administrators can approve applications"
    END IF

    // Step 2: Fetch application
    application = DATABASE.findOne("applications", WHERE application_id = applicationId)

    IF application is null THEN
        RETURN error "Application not found"
    END IF

    IF application.status != "Submitted" AND application.status != "Under Review" THEN
        RETURN error "Application cannot be approved (current status: " + application.status + ")"
    END IF

    // Step 3: Generate student number
    currentYear = GET_CURRENT_YEAR()

    // Get the last student number for this year
    lastStudent = DATABASE.findOne("students",
        WHERE student_number LIKE currentYear + "-%",
        ORDER BY student_number DESC,
        LIMIT 1)

    IF lastStudent exists THEN
        // Extract sequence number and increment
        lastSequence = EXTRACT_NUMBER(lastStudent.student_number)
        newSequence = lastSequence + 1
    ELSE
        newSequence = 1
    END IF

    studentNumber = currentYear + "-" + PAD_LEFT(newSequence, 4, "0")
    // Example: 2026-0001, 2026-0002, etc.

    // Step 4: Create student record
    studentRecord = {
        student_id: GENERATE_UUID(),
```

```
    user_id: application.user_id,  
    student_number: studentNumber,  
    id_number: application.id_number,  
    phone: application.phone,  
    address: application.address,  
    date_of_birth: application.date_of_birth,  
    program: application.program,  
    year_of_study: 1,  
    status: "Active",  
    enrollment_date: CURRENT_DATE(),  
    created_at: CURRENT_TIMESTAMP()  
}
```

```
DATABASE.insert("students", studentRecord)
```

```
// Step 5: Update user role from Applicant to Student
```

```
DATABASE.update("users",  
    SET role = "Student"  
    WHERE user_id = application.user_id)
```

```
// Step 6: Update application status
```

```
DATABASE.update("applications",  
    SET status = "Approved",  
        reviewed_by = adminUserId,  
        reviewed_at = CURRENT_TIMESTAMP()  
    WHERE application_id = applicationId)
```

```
// Step 7: Log the action
```

```
LOG_AUDIT("APPLICATION_APPROVED", adminUserId, "Application", applicationId, {  
    student_number: studentNumber  
}))
```

```
// Step 8: Send approval email to student
```

```
applicant = DATABASE.findOne("users", WHERE user_id = application.user_id)
```

```
SEND_EMAIL({  
    to: applicant.email,  
    subject: "Application Approved - Welcome to Richfield!",  
    body: "Congratulations! Your application has been approved.  
        Student Number: " + studentNumber + "  
        Program: " + application.program + "  
        You can now log in and register for courses."  
}))
```

```

// Step 9: Create in-app notification
CREATE_NOTIFICATION(application.user_id, "Application Approved",
    "Congratulations! Your application has been approved. Student #: " + studentNumber,
    "success", "/dashboard")

RETURN {
    success: true,
    studentNumber: studentNumber,
    message: "Application approved successfully"
}
END FUNCTION

```

4.6.4 Course Registration Process

```

FUNCTION registerForCourse(courseId, studentId):
    // Step 1: Fetch student record
    student = DATABASE.findOne("students", WHERE student_id = studentId)

    IF student is null OR student.status != "Active" THEN
        RETURN error "Student not found or not active"
    END IF

    // Step 2: Fetch course
    course = DATABASE.findOne("courses", WHERE course_id = courseId)

    IF course is null OR course.is_active = FALSE THEN
        RETURN error "Course not available"
    END IF

    // Step 3: Check registration period
    settings = GET_SYSTEM_SETTINGS()
    currentDate = CURRENT_DATE()

    IF currentDate < settings.registration_start_date THEN
        RETURN error "Registration has not opened yet"
    END IF

    IF currentDate > settings.registration_end_date THEN
        RETURN error "Registration period has closed"
    END IF

    // Step 4: Check if already registered for this course
    existingReg = DATABASE.findOne("registrations",

```

```

WHERE student_id = studentId
AND course_id = courseId
AND semester = settings.current_semester
AND status = "Registered")

IF existingReg exists THEN
    RETURN error "Already registered for this course"
END IF

// Step 5: Check course capacity
enrollmentCount = DATABASE.count("registrations",
    WHERE course_id = courseId
    AND semester = settings.current_semester
    AND status = "Registered")

IF enrollmentCount >= course.capacity THEN
    RETURN error "Course is full (capacity: " + course.capacity + ")"
END IF

// Step 6: Check prerequisites
IF course.prerequisites is not empty THEN
    prerequisiteCourses = PARSE_JSON(course.prerequisites)

    FOR EACH prereqCourseId IN prerequisiteCourses:
        completedPrereq = DATABASE.findOne("registrations",
            WHERE student_id = studentId
            AND course_id = prereqCourseId
            AND status = "Completed"
            AND grade IN ("A", "B", "C", "D"))

        IF completedPrereq is null THEN
            prereqCourse = DATABASE.findOne("courses", WHERE course_id = prereqCourseId)
            RETURN error "Prerequisite not met: " + prereqCourse.course_name
        END IF
    END FOR
END IF

// Step 7: Check schedule conflicts
studentRegistrations = DATABASE.findAll("registrations",
    WHERE student_id = studentId
    AND semester = settings.current_semester
    AND status = "Registered")

```



```

FOR EACH registration IN studentRegistrations:
    existingCourse = DATABASE.findOne("courses", WHERE course_id = registration.course_id)

    IF SCHEDULES_CONFLICT(course.schedule, existingCourse.schedule) THEN
        RETURN error "Schedule conflict with " + existingCourse.course_name
    END IF
END FOR

// Step 8: Check maximum credits
totalCredits = SUM(registrations.course.credits WHERE student_id = studentId
                    AND semester = current_semester AND status = "Registered")
totalCredits = totalCredits + course.credits

IF totalCredits > settings.max_credits_per_semester THEN
    RETURN error "Exceeds maximum credits per semester"
END IF

// Step 9: Create registration
registration = {
    registration_id: GENERATE_UUID(),
    student_id: studentId,
    course_id: courseId,
    semester: settings.current_semester,
    registration_date: CURRENT_TIMESTAMP(),
    status: "Registered",
    created_at: CURRENT_TIMESTAMP()
}

DATABASE.insert("registrations", registration)

// Step 10: Log the action
user = DATABASE.findOne("users", WHERE user_id = (
    SELECT user_id FROM students WHERE student_id = studentId))

LOG_AUDIT("COURSE_REGISTERED", user.user_id, "Registration", registration.registration_id, {
    course_code: course.course_code,
    course_name: course.course_name
})

// Step 11: Send confirmation email
SEND_EMAIL({
    to: user.email,
    subject: "Course Registration Confirmed",

```

```

        body: "You have successfully registered for " + course.course_code +
              " - " + course.course_name
    })

// Step 12: Create notification
CREATE_NOTIFICATION(user.user_id, "Registration Successful",
    "Successfully registered for " + course.course_name, "success")

RETURN {
    success: true,
    registrationId: registration.registration_id,
    message: "Successfully registered for course"
}
END FUNCTION

```

4.6.5 Student Number Generation Algorithm

```

FUNCTION generateStudentNumber():
    // Get current year
    currentYear = GET_CURRENT_YEAR() // e.g., 2026

    // Lock table to prevent race condition (multiple approvals at same time)
    DATABASE.BEGIN_TRANSACTION()
    DATABASE.LOCK_TABLE("students")

    TRY:
        // Find the highest student number for current year
        query = "SELECT student_number FROM students
                WHERE student_number LIKE '" + currentYear + "-%'"
                ORDER BY student_number DESC
                LIMIT 1"

        lastStudent = DATABASE.execute(query)

        IF lastStudent exists THEN
            // Extract sequence part (e.g., "2026-0042" -> "0042")
            lastNumber = lastStudent.student_number
            sequencePart = SUBSTRING(lastNumber, 5) // Get characters after "YYYY-"
            sequence = CONVERT_TO_INTEGER(sequencePart)
            newSequence = sequence + 1
        ELSE
            // First student of the year
            newSequence = 1
        END IF
    END IF

```

```

// Format: YYYY-####
// Pad sequence to 4 digits with leading zeros
sequenceStr = PAD_LEFT(newSequence, 4, "0")
studentNumber = currentYear + "-" + sequenceStr

// Example outputs:
// 2026-0001, 2026-0002, ... 2026-0999, 2026-1000

DATABASE.COMMIT_TRANSACTION()

RETURN studentNumber

CATCH error:
    DATABASE.ROLLBACK_TRANSACTION()
    THROW error "Failed to generate student number"
END TRY
END FUNCTION

```

4.6.6 Password Hashing and Verification

```

FUNCTION hashPassword(plainPassword):
    // Use bcrypt with salt rounds = 10
    // bcrypt automatically generates salt
    saltRounds = 10
    hashedPassword = BCRYPT.hash(plainPassword, saltRounds)

    // Returns something like:
    // "$2b$10$abcdefgijklmnopqrstuvwxyz1234567890ABCDEFGH"

    RETURN hashedPassword
END FUNCTION

FUNCTION verifyPassword(plainPassword, hashedPassword):
    // bcrypt.compare extracts salt from hash and compares
    isMatch = BCRYPT.compare(plainPassword, hashedPassword)

    RETURN isMatch // true or false
END FUNCTION

FUNCTION validatePasswordStrength(password):
    errors = []

    // Check length

```

```

IF LENGTH(password) < 8 THEN
    ADD errors "Password must be at least 8 characters"
END IF

// Check for uppercase
IF NOT CONTAINS_UPPERCASE(password) THEN
    ADD errors "Password must contain at least one uppercase letter"
END IF

// Check for lowercase
IF NOT CONTAINS_LOWERCASE(password) THEN
    ADD errors "Password must contain at least one lowercase letter"
END IF

// Check for number
IF NOT CONTAINS_NUMBER(password) THEN
    ADD errors "Password must contain at least one number"
END IF

// Check for special character (optional)
IF NOT CONTAINS_SPECIAL_CHAR(password) THEN
    ADD errors "Password should contain at least one special character"
END IF

// Check against common passwords list
commonPasswords = ["password", "123456", "qwerty", "admin", "welcome"]
IF password IN commonPasswords THEN
    ADD errors "Password is too common"
END IF

RETURN errors // Empty array if valid
END FUNCTION

```

4.6.7 File Upload Validation

```

FUNCTION validateFileUpload(file, maxSizeBytes, allowedTypes):
    // Step 1: Check if file exists
    IF file is null THEN
        RETURN error "No file provided"
    END IF

    // Step 2: Check file size (max 5MB = 5242880 bytes)
    IF file.size > maxSizeBytes THEN

```

```

    sizeMB = maxSizeBytes / 1024 / 1024
    RETURN error "File size exceeds maximum allowed size of " + sizeMB + "MB"
END IF

// Step 3: Check file type
fileExtension = GET_FILE_EXTENSION(file.name).toLowerCase()
mimeType = file.mimeType

validExtensions = ["pdf", "jpg", "jpeg", "png"]
validMimeTypes = ["application/pdf", "image/jpeg", "image/png"]

IF fileExtension NOT IN validExtensions OR mimeType NOT IN validMimeTypes THEN
    RETURN error "Invalid file type. Allowed: PDF, JPG, PNG"
END IF

// Step 4: Sanitize filename (remove special characters)
safeFilename = SANITIZE_FILENAME(file.name)

// Step 5: Generate unique filename to prevent collisions
uniqueId = GENERATE_UUID()
timestamp = CURRENT_TIMESTAMP()
newFilename = uniqueId + "_" + timestamp + "." + fileExtension

// Step 6: Determine storage path
storagePath = "/uploads/" + GET_CURRENT_YEAR() + "/" + GET_CURRENT_MONTH() + "/"
fullPath = storagePath + newFilename

// Step 7: Scan file for viruses (optional but recommended)
IF VIRUS_SCAN_ENABLED THEN
    scanResult = SCAN_FILE_FOR_VIRUSES(file)
    IF scanResult.infected = TRUE THEN
        LOG_SECURITY_EVENT("Malicious file upload attempt", file.name)
        RETURN error "File failed security scan"
    END IF
END IF

RETURN {
    valid: true,
    originalFilename: file.name,
    safeFilename: newFilename,
    storagePath: fullPath,
    fileSize: file.size,
    mimeType: mimeType
}
END FUNCTION

```

4.7 Interface Design (Menu, Input, Output Design)

This section describes the user interface design - what users will see and interact with.

4.7.1 Main Menu Structure

For Students:

EduHub Logo	[Student Name] ▼
Dashboard	
Courses	
└─ Available Courses	
└─ My Registered Courses	
└─ Course History	
Profile	
└─ Personal Information	
└─ Emergency Contacts	
└─ Change Password	
Applications	
└─ New Application	
└─ Application Status	
Notifications (3)	
Logout	

For Administrators:

EduHub Logo	[Admin Name] ▼
Dashboard	
Applications	
└─ Pending Applications	
└─ All Applications	
└─ Bulk Actions	
Course Management	
└─ All Courses	
└─ Create Course	
└─ Assign Lecturers	
User Management	
└─ All Users	
└─ Role Management	
└─ Deactivate Users	
System Settings	

Registration Periods System Configuration Email Templates Reports Enrollment Reports Application Statistics System Usage Audit Logs Notifications Logout

For Lecturers:

EduHub Logo	[Lecturer Name] ▼
Dashboard My Courses Class Rosters Announcements Post Announcement View All Profile Notifications Logout	

4.7.2 Key Screen Designs

Login Page

Email Address student@example.com Password ☐ Remember me Login Button Forgot Password? Don't have an account? Register here
--

Application Form (Multi-step)

Step 1: Personal Information

New Application - Step 1 of 3

● Personal Information ○ Program Selection ○ Upload

First Name *

John

Last Name *

Doe

ID Number *

9001015800084

Date of Birth *

 01/15/1990

Email *

john.doe@example.com

Phone Number *

+27 12 345 6789

Address *

123 Main Street, Johannesburg, 2000

[Save as Draft]

[Next: Program Selection].

Student Dashboard

Screen 1 — Student Dashboard

EduHub

Dashboard

Courses

Profile

Notifications

Logout

Welcome back, John Doe | Student #2026-0042

My Courses

CS101: Intro to CS

MAT201: Calculus I

ENG101: English

Total: 3 courses (12 credits)

Notifications

 Registration opens tomorrow

 Application approved!

[View All \(5\)](#)

Quick Actions

[Register for Courses](#)

[Update Profile](#)

Important Dates

 **Registration Opens:** June 10, 2026

 **Registration Closes:** June 20, 2026

 **Add/Drop Deadline:** July 1, 2026

 **Semester Starts:** July 15, 2026

Screen 2 — Course Registration

EduHub

Dashboard

Courses

Profile

Notifications

Logout

Available Courses — Fall 2026

Search: _____

Department: All ▼

CS101 — Introduction to Computer Science ● 30/50 enrolled

Credits: 4 | **Capacity:** 30/50 | **Lecturer:** Dr. Smith
Schedule: Mon/Wed 9:00 – 10:30
Prerequisites: None

[\[View Details\]](#)

[\[Register \]](#)

CS102 — Data Structures ● FULL

Credits: 4 | **Capacity:** 50/50 | **Lecturer:** Dr. Jones
Schedule: Tue/Thu 11:00 – 12:30
Prerequisites: CS101 ⚠ NOT MET

[\[View Details\]](#)

[\[Register \]](#)
(disabled)

MAT201 — Calculus I ● 15/40 enrolled

Credits: 4 | **Capacity:** 15/40 | **Lecturer:** Prof. Lee
Schedule: Mon/Wed 14:00 – 15:30
Prerequisites: None

[\[View Details\]](#)

[\[Register \]](#)

Currently Registered: 2 courses (8 credits)

Remaining Credits Available: 12

4.7.3 Input Design Standards

Form Field Standards:

- **Required Fields:** Marked with red asterisk (*)
- **Field Validation:** Real-time validation with error messages below field
- **Input Formats:**
 - Dates: YYYY-MM-DD or DD/MM/YYYY with date picker
 - Phone: +27 XX XXX XXXX format
 - Email: Standard email format validation
 - ID Number: 13 digits for SA ID numbers
 - Student Number: YYYY-#### format (read-only, generated)

Error Message Format:

✖ Email is required

✗ Invalid email format

✗ ID number must be exactly 13 digits

 Valid input

Button States:

- **Primary Action:** Blue background, white text
- **Secondary Action:** Gray background, dark text
- **Disabled:** Gray background, light gray text, no hover
- **Hover:** Slightly darker background
- **Loading:** Show spinner, disable button

4.7.4 Output Design

Report Format (Enrollment Report Example):

RICHFIELD EDUHUB - ENROLLMENT REPORT	
Semester: Fall 2026	
Generated: 2026-06-08 14:30	
Generated by: Admin User	
SUMMARY STATISTICS	
Total Students: 1,247	
Total Courses: 85	
Total Registrations: 4,523	
Average Credits per Student: 15.2	

ENROLLMENT BY PROGRAM

Computer Science: 287 students (23%)
Business Administration: 312 students (25%)
Engineering: 198 students (16%)
Other Programs: 450 students (36%)

TOP 5 MOST ENROLLED COURSES

1. CS101 - Intro to CS 50/50 (100%)
2. MAT201 - Calculus I 48/50 (96%)
3. ENG101 - English 45/50 (90%)
4. BUS201 - Marketing 42/45 (93%)
5. PHY101 - Physics 40/40 (100%)

[\[Export to PDF\]](#) [\[Export to CSV\]](#) [\[Print\]](#)

Notification Formats:

Success Notification:

 Success

Application submitted successfully!

Application ID: APP-2026-1234

[\[View Application\]](#) [\[Dismiss\]](#)

Error Notification:

 Error

Failed to register for course

Reason: Course is full

[\[View Available Courses\]](#) [\[Dismiss\]](#)

Warning Notification:

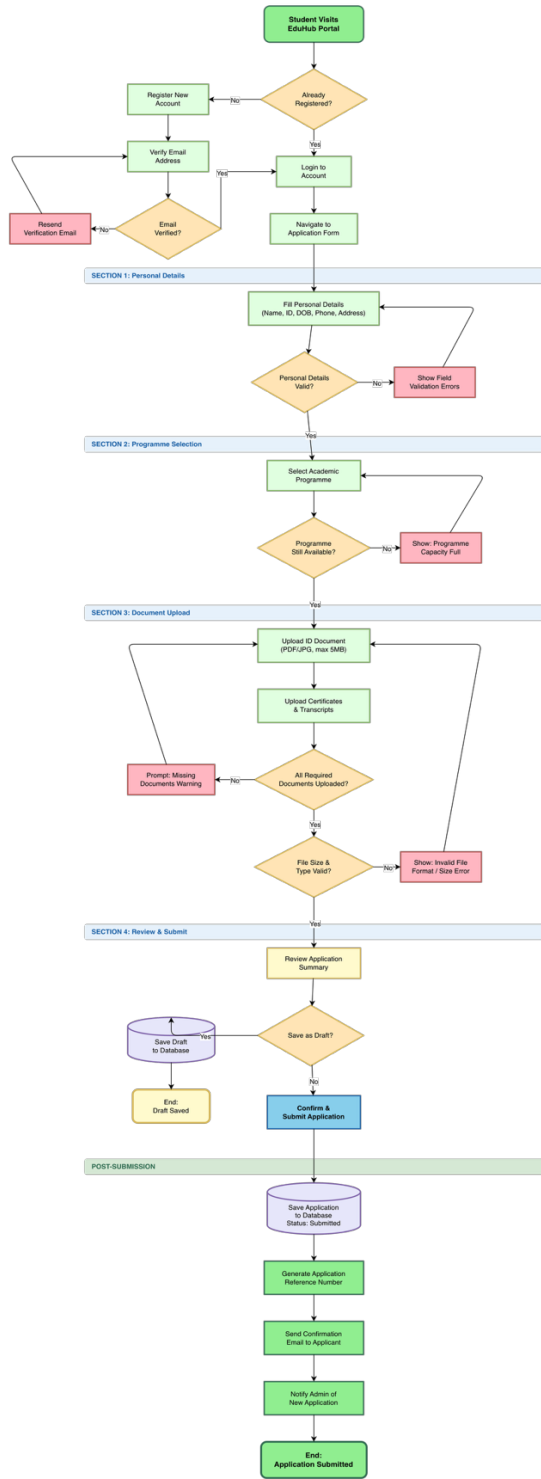
 Warning

Registration closes in 2 days!

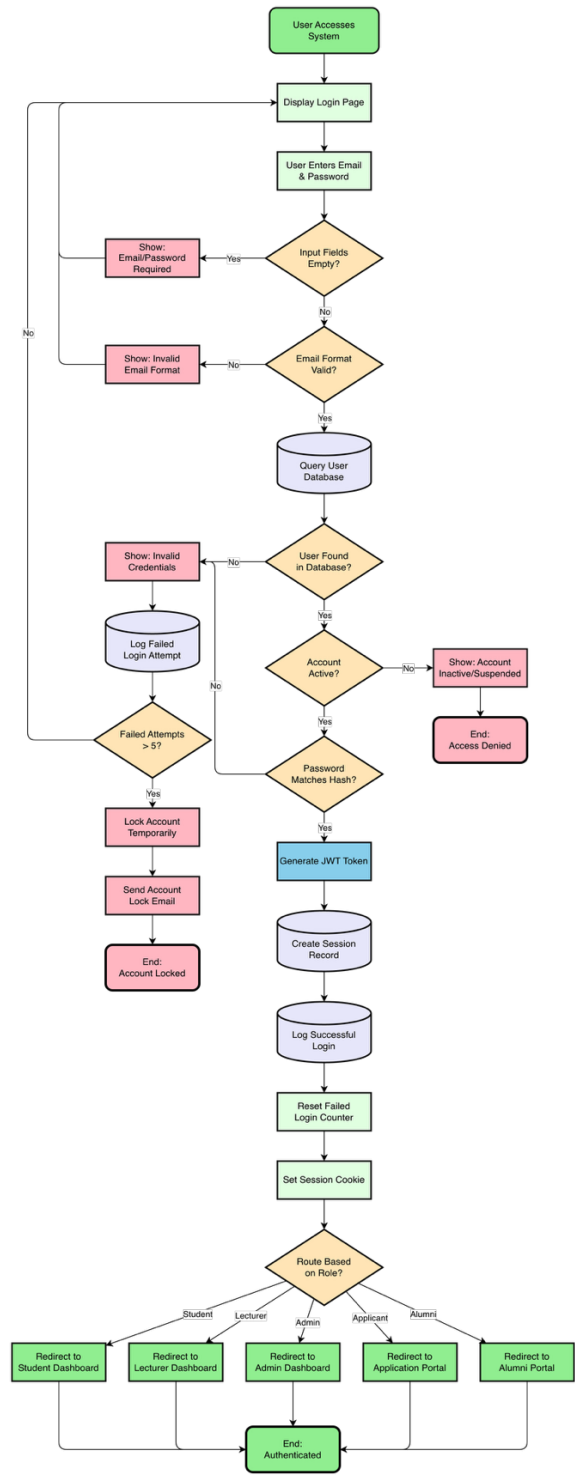
[\[Register Now\]](#) [\[Dismiss\]](#)

4.7.5 Flow Charts

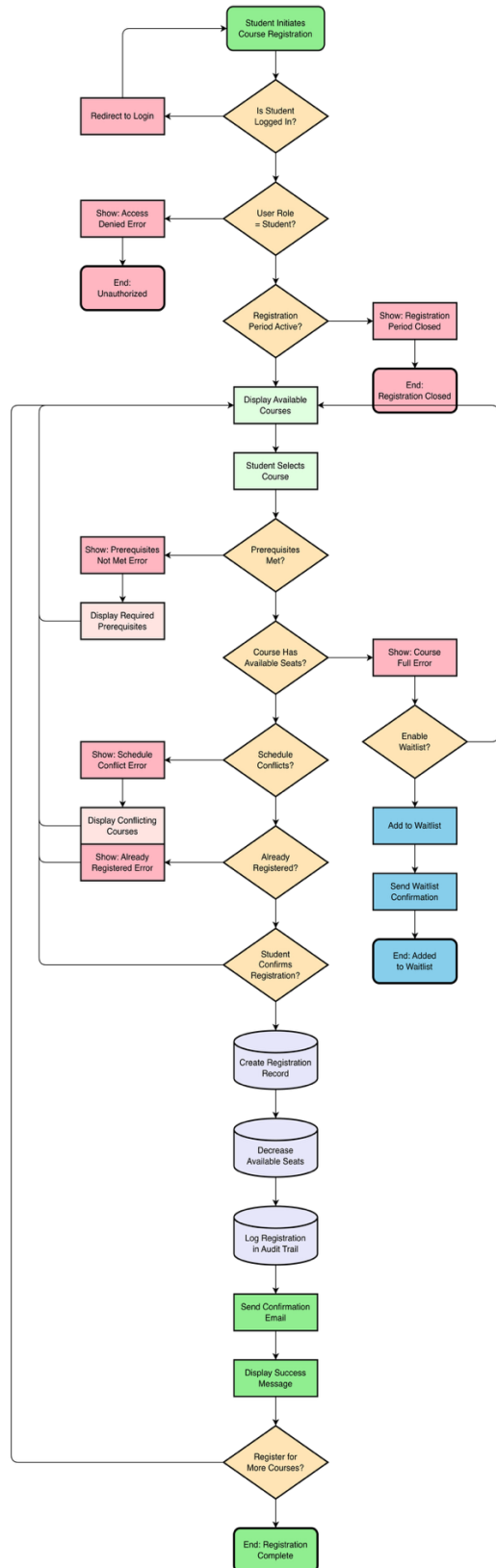
Student Application



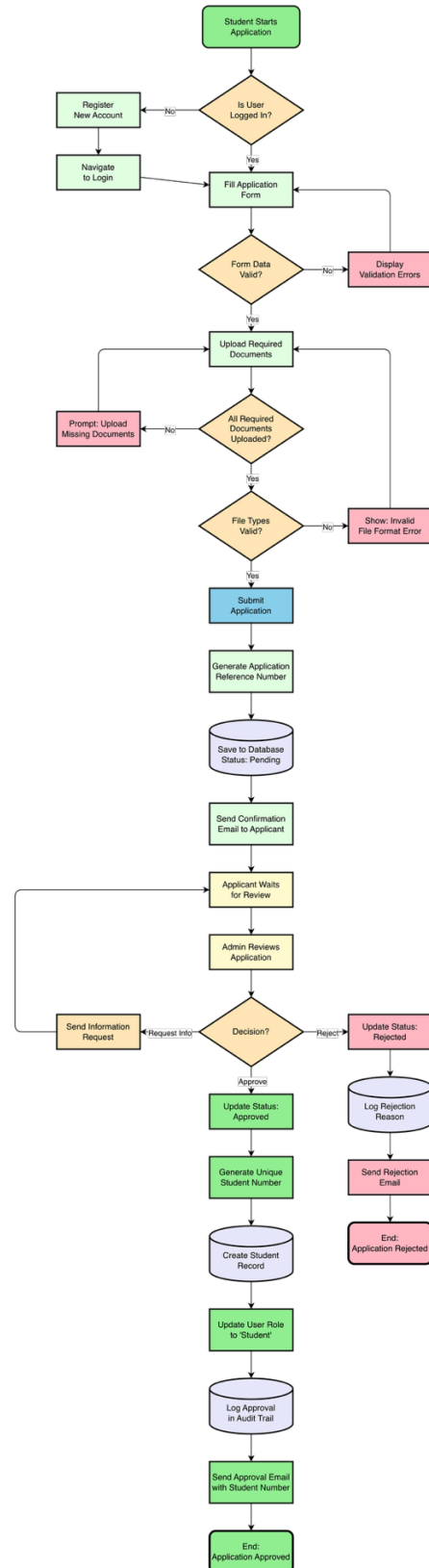
User Authentication



Registration

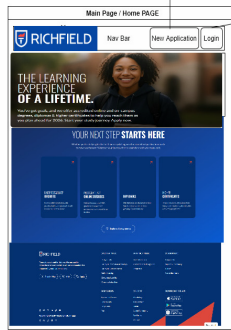


Application Approval



MainFlow

1. this is the layout for the home page to create link to new App and login The links at bottom, for now we use Richfield



Text

RICHFIELD APPLICATION FORM - FILLED

Personal Details

Student Personal Details

Name: [Name]
Age: [Age]
Gender: [Gender]
Date of Birth: [Date]
Address: [Address]
Contact Details: [Contact]

Qualification Selection

Qualification: [Qualification]
Level: [Level]
Subject: [Subject]

Documents

Required Documents: [Documents]

Ts and Cs

Terms and Conditions: [Terms]

Account Payer Details

Parent/Responsible for Account

Name: [Name]
Address: [Address]
Contact Details: [Contact]

Qualification Selection

Qualification: [Qualification]
Level: [Level]
Subject: [Subject]

Documents

Required Documents: [Documents]

Ts and Cs

Terms and Conditions: [Terms]

At the Bottom We have submit application Button

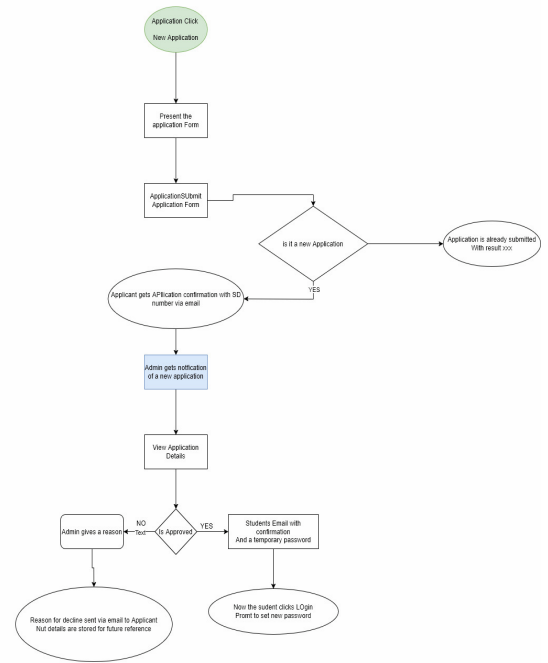
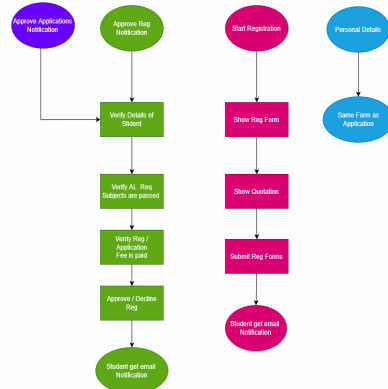
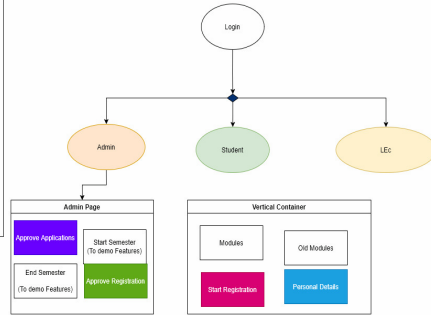
Once Button is pressed the Appikats will receive confirmation that Application has been submitted

AN Email is sent to student with a reference number, which is going to be their student number

Users

Type	Prefix	Year	Start	Randomiser
Student	SD	2561 / 2567	3254332	
Admin	AM			
Lecturer	LT			

8025912323424



4.8 Security and Backup Design

This section describes how we'll keep the system and data secure, and how we'll recover from failures.

4.8.1 Security Design

Authentication Security

Password Security:

- Passwords hashed using **bcrypt** with salt rounds = 10
- Never store passwords in plain text
- Password requirements:
 - Minimum 8 characters
 - At least 1 uppercase letter
 - At least 1 lowercase letter
 - At least 1 number
 - At least 1 special character (recommended)
- Failed login attempts tracked:
 - After 5 failed attempts, account temporarily locked for 15 minutes
 - After 10 failed attempts, account locked until admin unlocks

Token Security:

- JWT (JSON Web Tokens) used for session management
- Tokens stored in httpOnly cookies (not accessible via JavaScript)
- Token expiry: 24 hours
- Refresh tokens not implemented in MVP (future enhancement)
- Tokens include:
 - userId
 - email
 - role
 - issued at timestamp
 - expiry timestamp

Multi-Factor Authentication (MFA):

- Optional for users, required for admins
- Uses TOTP (Time-based One-Time Password)
- Compatible with Google Authenticator
- Authy - Backup codes provided (10 single-use codes)

Authorization Security

Role-Based Access Control (RBAC):

Role	Permissions
Applicant	- Submit applications - View own application status - Upload documents
Student	- All Applicant permissions - Register for courses - View registered courses - Drop courses - Update profile - View notifications
Lecturer	- View assigned courses - View class rosters - Post announcements - Export student lists
Admin	- All permissions - Approve/reject applications - Create/edit/delete courses - Manage users - Configure system settings - View audit logs - Generate reports
Alumni	- View transcript (future) - Update contact info (future)

API Endpoint Protection:

Every API request must include:

1. Valid JWT token in Authorization header
2. Role check: Does user's role have permission?
3. Ownership check: Can user access this specific resource?

Example:

GET /api/students/12345/profile

- Check: Is user logged in? (valid token)
- Check: Is user's role Student/Admin?
- Check: Is userId = 12345 OR is user Admin?

Data Security

Encryption:

- **Data in Transit:** All communications use HTTPS/TLS 1.3
- **Data at Rest:**
 - Passwords: Bcrypt hashed
 - Sensitive fields (ID numbers): AES-256 encryption (future enhancement)
 - Database backups: Encrypted before storage

SQL Injection Prevention:

- **Use Sequelize ORM**
 - automatically parameterizes queries
 - Never concatenate user input into SQL queries
 - Example (WRONG): `sql query = "SELECT * FROM users WHERE email = '" + userInput + "'"`
 - Example (CORRECT): `javascript User.findOne({ where: { email: userInput } })`
- **XSS (Cross-Site Scripting) Prevention:**
 - HTML/Javascript automatically escapes output (prevents XSS)
 - Sanitize HTML content if allowing rich text
 - Set CSP (Content Security Policy) headers
 - Validate and sanitize all user inputs

CSRF (Cross-Site Request Forgery) Prevention:

- Use CSRF tokens for state-changing operations
- SameSite cookie attribute set to "Strict"
- Verify Origin/Referer headers

File Upload Security:

- Validate file type (check MIME type AND extension)
- Limit file size (max 5MB)
- Scan files for viruses (ClamAV integration recommended)
- Store files outside web root
- Generate random filenames to prevent path traversal
- Never execute uploaded files

Network Security

Firewall Rules:

- ALLOW: Port 443 (HTTPS) from anywhere
- ALLOW: Port 80 (HTTP) - redirect to 443
- DENY: Port 5432 (PostgreSQL) from internet
- ALLOW: Port 5432 from application servers only
- DENY: All other ports

DDoS Protection:

- Use Cloudflare or similar CDN
- Rate limiting: Max 100 requests per minute per IP
- Implement request throttling

Security Headers:

- Strict-Transport-Security: max-age=31536000; includeSubDomains
- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- X-XSS-Protection: 1; mode=block
- Content-Security-Policy: default-src 'self'

4.8.2 Backup Design

Backup Strategy

Database Backups:

- **Full Backups:**
 - Frequency: Daily at 2:00 AM (low traffic time)
 - Retention: 30 days
 - Storage location: Separate server/cloud storage
 - Encryption: Yes, AES-256
 - Compression: Yes, gzip
- **Incremental Backups:**
 - Frequency: Every 6 hours
 - Retention: 7 days
 - Only backs up changes since last backup
- **Transaction Log Backups:**
 - Frequency: Every hour
 - Retention: 24 hours
 - Enables point-in-time recovery

Backup Script (Pseudocode)

```
#!/bin/bash
# Daily backup script

# Set variables
DATE=$(date +%Y-%m-%d)
BACKUP_DIR="/backups/database"
DB_NAME="eduhub"
DB_USER="backup_user"
```

```

# Create backup
pg_dump -U $DB_USER $DB_NAME | gzip > $BACKUP_DIR/eduhub_$(date +%Y%m%d).sql.gz

# Encrypt backup
openssl enc -aes-256-cbc -in $BACKUP_DIR/eduhub_$(date +%Y%m%d).sql.gz \
  -out $BACKUP_DIR/eduhub_$(date +%Y%m%d).sql.gz.enc -k $ENCRYPTION_KEY

# Upload to cloud storage
aws s3 cp $BACKUP_DIR/eduhub_$(date +%Y%m%d).sql.gz.enc s3://eduhub-backups/

# Delete local backup older than 7 days
find $BACKUP_DIR -name "*.enc" -mtime +7 -delete

# Log completion
echo "Backup completed: $(date)" >> /var/log/backup.log

```

File Storage Backups:

- Frequency: Daily
- Method: Rsync to backup server
- Retention: 30 days
- Includes: Uploaded documents (applications, profile photos)

Code Backups:

- Version control: Git repository
- Remote backup: GitHub/GitLab
- Branch protection: Main branch protected
- Backup frequency: Continuous (every git push)

Disaster Recovery Plan

Recovery Time Objective (RTO): 4 hours - Maximum acceptable downtime after disaster

Recovery Point Objective (RPO): 1 hour - Maximum acceptable data loss

Disaster Recovery Steps:

- **Detect Failure:**
 - Monitoring system alerts (uptime monitoring, error rate spike)
 - Manual notification from users
- **Assess Damage:**
 - Is it hardware failure, software bug, or data corruption?
 - What data/services are affected?
- **Activate Disaster Recovery:**
 - Switch to backup server if available
 - Restore from latest backup
- **Database Restore Procedure:**

Step 1: Download encrypted backup from cloud

```
aws s3 cp s3://eduhub-backups/eduhub_2026-06-07.sql.gz.enc /restore/
```

Step 2: Decrypt backup

```
openssl enc -d -aes-256-cbc -in /restore/eduhub_2026-06-07.sql.gz.enc \
  -out /restore/eduhub_2026-06-07.sql.gz -k $ENCRYPTION_KEY
```

Step 3: Decompress

```
gunzip /restore/eduhub_2026-06-07.sql.gz
```

Step 4: Restore to database

```
psql -U postgres $DB_NAME < /restore/eduhub_2026-06-07.sql
```

Step 5: Verify data integrity

Run verification queries

Step 6: Bring system online

- **Verify Restore:**
 - Check critical functions (login, registration, etc.)
 - Verify recent data exists
 - Test all major workflows
- **Communicate:**
 - Notify users system is back online
 - Inform about any data loss (e.g., “Last 2 hours of data recovered from backup”)
- **Post-Mortem:**
 - Document what went wrong
 - Implement measures to prevent recurrence

Backup Testing

Monthly Backup Test:

- Restore latest backup to test environment
- Verify all data intact
- Test application functionality
- Document results
- Time the restore process

Annual Disaster Recovery Drill:

- Simulate complete system failure
- Practice full recovery procedure
- Involve all team members
- Update DR plan based on findings

Design Summary

What We've Designed:

- **System Architecture** - Three-tier architecture with HTML/Javascript frontend, Node.js backend, PostgreSQL database
- **Technology Stack** - Modern, scalable technologies suitable for Richfield's needs
- **Database Design** - 10 tables with detailed schemas, relationships, and constraints
- **Program Logic** - Pseudocode for all critical processes (login, registration, approvals)
- **User Interface** - Screen designs, menu structures, input/output formats
- **Security Measures** - Authentication, authorization, encryption, input validation
- **Backup Strategy** - Daily backups, disaster recovery plan, tested procedures

Design Principles Applied:

- **User-centered:** Designed for actual users (students, lecturers, admin)
- **Secure by design:** Security built in from the start
- **Scalable:** Can grow from 1,000 to 10,000+ students
- **Maintainable:** Clear separation of concerns, well-documented
- **Testable:** Modular design enables thorough testing